

MACHINE LEARNING

LAB MANUAL

IFETCE R-2023



IFET COLLEGE OF ENGINEERING

[An Autonomous Institution]

Department of Computer Science and Engineering

SUBJECT CODE : 23CS5L02
YEAR/SEM: III/V

Prepared by:

Dr.K.Sathyabama ,
Mrs.C.Manimegalai,
Mrs. E.Jothi

Verified by:

Ms.M.Martina Jose Mary

Approved by:

Mr.D.Raghu Raman

DEPARTMENT OF CSE

VISION AND MISSION OF THE DEPARTMENT

VISION

To produce demand driven, quality conscious and globally recognized computer professionals through education, innovation and collaborative research.

MISSION

- To develop graduates of world-class technical competence in the area of Computer Science and Engineering, with necessary skills to solve real world problems.
- To establish a center of excellence in partnership with industries, research institutions, and other organizations to meet the evolving needs of society.
- To inculcate ethical values and the spirit of entrepreneurship

PROGRAMME OUTCOMES (PO)

- ❖ POs On the completion of the program the graduates are expected to know or develop the abilities defined through the Program Outcomes. The Program Outcomes of Computer Science and Engineering program is given below.
- PO 1:** Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
- PO 2:** Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
- PO 3:** Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
- PO 4:** Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
- PO 5:** Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitation.
- PO 6:** The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
- PO 7:** Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

- PO 8:** Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
- PO 9:** Individual and teamwork: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
- PO 10:** Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
- PO 11:** Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply the set to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
- PO 12:** Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAMME SPECIFIC OUTCOMES (PSO)

On the completion of the program the graduates are expected to know or develop the abilities defined through the Program Specific Outcomes.

The Program Specific Outcomes of Computer Science and Engineering program is given below.

PSO 1 Apply the fundamental knowledge of Computer Science & Engineering to design and develop software applications.

PSO 2 Develop competency in software development, software testing, data storage, computing, and business intelligence fields.

PSO 3 Ability to design and develop innovative solutions that meet rising global demands using the latest technologies.

PROGRAM EDUCATIONAL OBJECTIVES (PEO)

PEO 1 Graduates will be equipped to take on technical and managerial roles, covering areas such as design, development, problem-solving, and production support within the software industry and R&D sectors.

PEO 2 Graduates will be able to successfully pursue higher education at reputed institutions.

PEO 3 Graduates will act as ethical and socially responsible solution providers and entrepreneurs within Computer Science and other engineering disciplines.

SYLLABUS

23IT5L02	MACHINE LEARNING LABORATORY	L	T	P	C
		0	0	3	1.5

LEARNING OBJECTIVES:

The faculty will enhance the skills of the students to :

- Implement and evaluate machine learning algorithms like decision trees, support vector machines, and k-nearest neighbors for classification tasks.
- Apply advanced techniques such as facial recognition and sentiment analysis using artificial neural networks and random forest algorithms.
- Gain proficiency in using Amazon SageMaker to build, train, and deploy machine learning models.
- Develop and implement regression algorithms like Locally Weighted Regression and construct Bayesian networks for data modeling.
- Work on real-world applications like spam detection, online fraud detection, and medical data classification through practical projects.

LIST OF EXPERIMENTS

1. Implement the concept of decision trees with suitable data set from real world problem and classify the data set to produce new sample.
2. Detecting Spam mails using Support vector machine.
3. Implementation of facial recognition application with artificial neural network.
4. Study and implement amazon toolkit: Sagemaker.
5. Implement k-nearest neighbours classification.
6. Implementation of non-parametric Locally Weighted Regression algorithm for data-point fitting.
7. Implement sentiment analysis using random forest optimization algorithm.
8. Write a program to construct a Bayesian network considering medical data.
9. Implement online fraud detection.
10. Mini-project.

TOTAL : 45 PERIODS

COURSE OUTCOMES : After successful completion of this course, students will able to

- | | |
|---------------|--|
| C308.1 | Implement machine learning algorithms (e.g., decision trees, SVM, k-nearest neighbors) for classification tasks and analyze real-world datasets. |
| C308.2 | Develop facial recognition and sentiment analysis applications using artificial neural networks and random forest algorithms. |
| C308.3 | Utilize Amazon SageMaker to build and deploy machine learning models efficiently. |
| C308.4 | Implement Locally Weighted Regression and Bayesian networks to solve data fitting and classification problems. |
| C308.5 | Design and develop applications for spam detection and online fraud detection, applying machine learning techniques to real-world problems. |

CO- PO, PSO MAPPING

POs & PSOs COs	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3
C308.1	3	2	2	1	2	-	1	1	2	1	1	1	3	2	2
C308.2	3	2	2	1	2	-	1	1	2	1	1	2	3	2	2
C308.3	3	3	3	3	2	-	1	1	2	1	1	2	3	3	3
C308.4	3	3	3	3	2	-	1	1	2	1	1	2	3	3	3
C308.5	3	3	2	2	2	-	1	1	2	1	1	2	3	3	3
AVG	3	3	2	3	2	-	1	1	2	1	1	2	3	3	3
Correlation levels: 1: Slight (Low) 2: Moderate (Medium) 3: Substantial (High)															

INDEX TABLE OF CONTENTS

S.No	Date of Expt.	Name of the Experiment	Page No	Marks			Staff Initial
				Viva (10)	Expt. (10)	Total (20)	
1.		Implement the concept of decision trees with suitable data set from real world problem and classify the data set to produce new sample.	7				
2.		Detecting Spam mails using Support vector machine.	13				
3.		Implementation of facial recognition application with artificial neural network.	16				
4.		Study and implement amazon toolkit: Sagemaker	22				
5.		Implement k-nearest neighbours classification.	30				
6.		Implementation of non-parametric Locally Weighted Regression algorithm for data-point fitting.	35				
7.		Implement sentiment analysis using random forest optimization algorithm.	37				
8.		Write a program to construct a Bayesian network considering medical data.	42				
9.		Implement online fraud detection.	47				
10.		Mini-project.					
Virtual Lab							
11.		Implementation of different method os Defuzzification process	55				
12.		Understanding the concepts of Perceptron Learning Rule	57				
Content Beyond The Syllabus							
13.		Implementation Back propagation Algorithm using ANN	59				
14.		Demonstrate Naïve Bayesian Classifier Machine Learning	63				
Total Marks							

Ex. No. : 1 Implement the concept of decision trees with suitable data set from real world problem and classify the data set to produce new sample.

PROBLEM STATEMENT:

Write a program to implement the concept of decision trees with suitable data set from real world problem and classify the data set to produce new sample..

AIM:

To implement the concept of decision trees with suitable data set from real world problem and classify the data set to produce new sample.

PROCEDURE:

Step 1: Import the required libraries Step 2: Load Iris dataset

Step 3: Explore the dataset

Step 4: Perform pair plotting to view relationship between the features present in the data

Step 5: Split data into train and test set

Step 6: Train the decision tree classifier

Step 7: Test the model trained on test set

7a. Perform Evaluation on Test set 7b. Plot Confusion Matrix

Step 8: Visualize the Decision Tree

Step 9: Feeding new data to the classifier to predict the right class

PROGRAM:

Import the required libraries

```
import sklearn.datasets as datasets
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report,
plot_confusion_matrix
from sklearn import tree
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline
```

Load Iris dataset

```
from google.colab import files
upload = files.upload()
```

Output:

Choose Files Iris.csv
 • Iris.csv(text/csv) - 5107 bytes, last modified: 7/22/2022 - 100% done
 Saving Iris.csv to Iris.csv

Load data from CSV

```
iris_df = pd.read_csv("/content/Iris.csv")
iris_df.head()
```

Output:

Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
----	---------------	--------------	---------------	--------------	---------

0	1	5.1	3.5	1.4	0.2	Íris-setosa
1	2	4.9	3.0	1.4	0.2	Íris-setosa
2	3	4.7	3.2	1.3	0.2	Íris-setosa
3	4	4.6	3.1	1.5	0.2	Íris-setosa
4	5	5.0	3.6	1.4	0.2	Íris-setosa

Explore the dataset

```
iris_df.isnull().any()
```

Output:

```
Id          False
SepalLengthCm  False
SepalWidthCm  False
PetalLengthCm  False
PetalWidthCm  False
Species      False
dtype: bool
```

```
iris_df.dtypes
```

Output:

```
Id          int64
SepalLengthCm  float64
SepalWidthCm  float64
PetalLengthCm  float64
PetalWidthCm  float64
Species      object
dtype: object
```

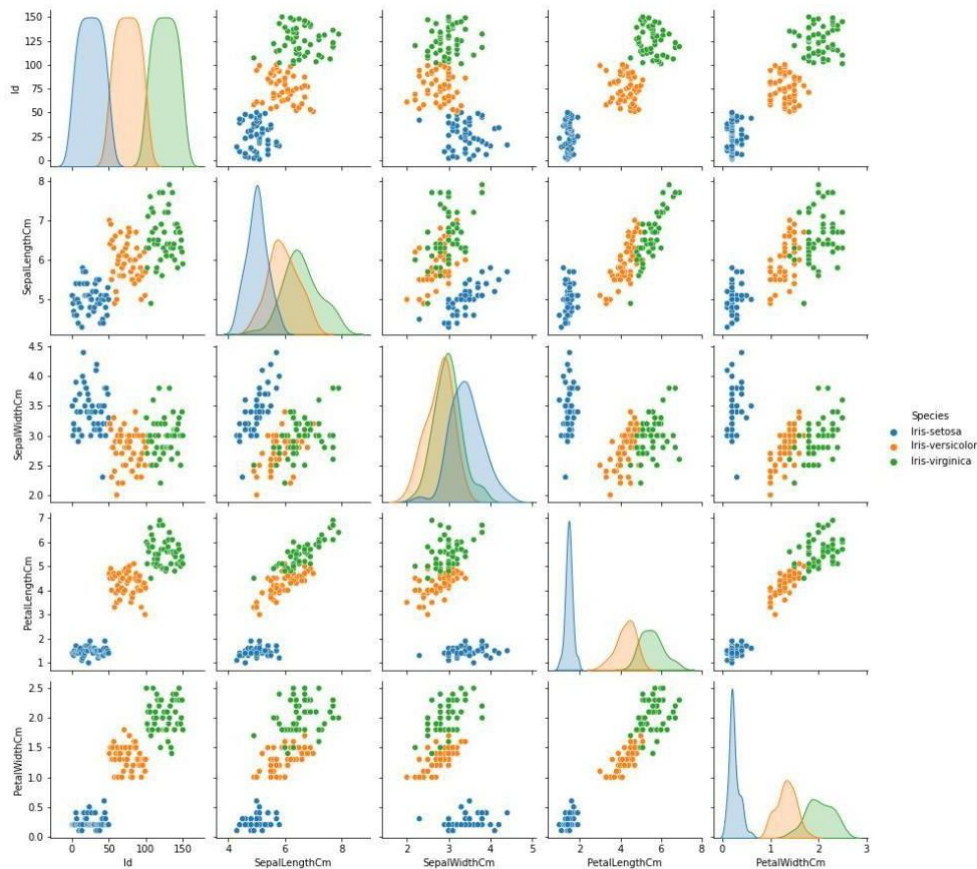
```
iris_df.describe()
```

Output:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	75.500000	5.843333	3.054000	3.758667	1.198667
std	43.445368	0.828066	0.433594	1.764420	0.763161
min	1.000000	4.300000	2.000000	1.000000	0.100000
25%	38.250000	5.100000	2.800000	1.600000	0.300000
50%	75.500000	5.800000	3.000000	4.350000	1.300000
75%	112.750000	6.400000	3.300000	5.100000	1.800000
max	150.000000	7.900000	4.400000	6.900000	2.500000

Perform pair plotting to view relationship between the features present in the data

```
sns.pairplot(iris_df, hue='Species')
plt. savefig("exercise2a.png")
files. download("exercise2a.png")
```

Output:**Split data into train and test set**

```
X = iris_df[['SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm',
'PetalWidthCm']].values = iris_df['Species'].values
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.9, random_state=1)
```

Train the decision tree classifier

```
#scikit learn decision tree
model trainingclf =
tree.DecisionTreeClassifier()
clf = clf.fit(X_train, y_train)
```

Test the model trained

```
on test setprediction =
clf.predict(X_test)
prediction
```

Output:

```
array(['Iris-setosa', 'Iris-versicolor', 'Iris-versicolor', 'Iris-setosa',
'Iris-virginica', 'Iris-versicolor', 'Iris-virginica',
'Iris-setosa', 'Iris-setosa', 'Iris-virginica', 'Iris-versicolor',
'Iris-setosa', 'Iris-virginica', 'Iris-versicolor',
'Iris-versicolor'], dtype=object)
```

Perform Evaluation on Test set

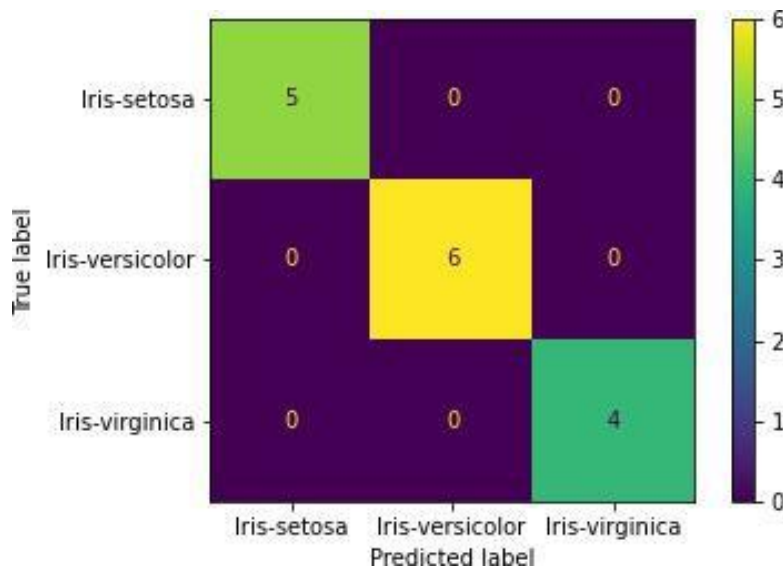
evaluation for multi class classification

`print(classification_report(y_test, prediction))`**Output:**

	precision	recall	f1-score	support
Iris-setosa	1.00	1.00	1.00	5
Iris-versicolor	1.00	1.00	1.00	6
Iris-virginica	1.00	1.00	1.00	4
accuracy			1.00	15
macro avg	1.00	1.00	1.00	15
weighted avg	1.00	1.00	1.00	15

Plot Confusion Matrix`plot_confusion_matrix(clf,``X_test, y_test)plt.``savefig("exercise2b.png")``files.download("exercise2b.png")`**Output:**`/usr/local/lib/python3.7/dist-``packages/sklearn/utils/deprecation.py:87:`

FutureWarning:

Function `plot_confusion_matrix` is deprecated; Function`'plot_confusion_matrix'` is deprecated in 1.0 and will be removed in 1.2. Use one of the class methods:`ConfusionMatrixDisplay.from_predictions` or`ConfusionMatrixDisplay.from_estimator.``warnings.warn(msg, category=FutureWarning)`**Visualize the Decision Tree**`!pip install pydotplus`

!apt-get install graphviz -y

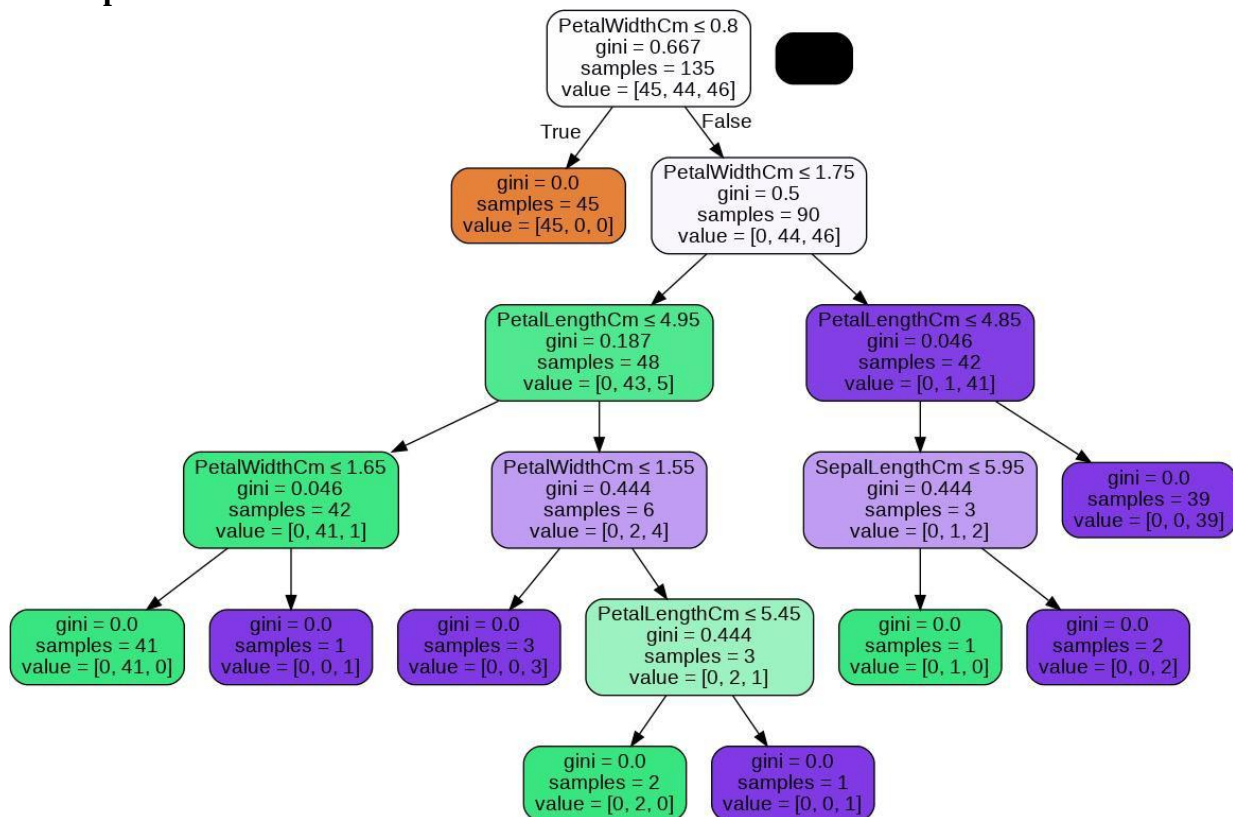
```
Looking in indexes: https://pypi.org/simple, https://us-python.pkg.dev/colab-wheels/public/simple/
Requirement already satisfied: pydotplus in /usr/local/lib/python3.7/dist-packages (2.0.2)
Requirement already satisfied: pyparsing>=2.0.1 in /usr/local/lib/python3.7/dist-packages (from pydotplus) (3.0.9)
Reading package lists... Done
Building dependency tree
Reading state information... Done
graphviz is already the newest version (2.40.1-2).
The following package was automatically installed and is no longer required:
  libnvidia-common-460
Use 'apt autoremove' to remove it.
0 upgraded, 0 newly installed, 0 to remove and 49 not upgraded.
```

```
from six import StringIO
from IPython.display import Image
from sklearn.tree import export_graphviz, pydotplus
```

```
# Visualize the graphdot_data = StringIO()
export_graphviz(clf, out_file=dot_data, feature_names=['SepalLengthCm',
'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm'],
                filled=True, rounded=True,
                special_characters=True)
graph = pydotplus.graph_from_dot_data(dot_data.getvalue())
Image(graph.create_png())
plt.savefig("exercise2c.png")
```

files.download("exercise2c.png")

Output:



Feeding new data to the classifier to predict the right class

SepalLengthCm = 4.8

SepalWidthCm = 2.9

PetalLengthCm = 1.3

```
PetalWidthCm = 0.2
```

```
x = [[SepalLengthCm, SepalWidthCm, PetalLengthCm,
```

```
PetalWidthCm]]res = clf.predict(x)
```

```
print("The class predicted is --> " + str(*res))
```

Output:

```
The class predicted is --> Iris-setosa
```

RESULT:

Thus the concept of decision trees with suitable data set from real world problem and to classify the data set to produce new sample was implemented successfully.

Ex. No. : 2**Detecting Spam mails using Support vector machine.****PROBLEM STATEMENT:**

Write a program to demonstrate the working of the Spam mail detection using Support vectormachine

AIM:

To implement Spam mail detection using Support vector machine.

PROCEDURE:

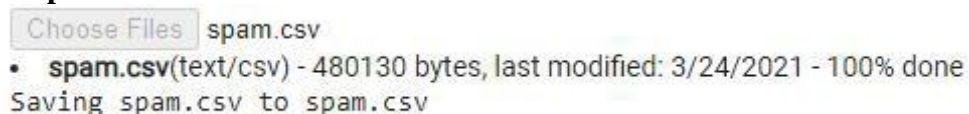
- Step 1: Import the
required librariesStep 2:
Load the spam dataset
Step 3: Explore the dataset
Step 5: Split data into training and testing set
Step 6: Converting text into integer using
CountVectorizer()Step 7: Applying SVM
algorithm
Step 8: Find the Accuracy

PROGRAM:**Import the required libraries**

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.feature_extraction.text
import CountVectorizerfrom
sklearn.model_selection import GridSearchCV
```

Load the spam dataset

```
from google.colab
import filesupload = files.upload()
```

Output:

Choose Files spam.csv
• spam.csv(text/csv) - 480130 bytes, last modified: 3/24/2021 - 100% done
Saving spam.csv to spam.csv

```
dataset = pd.read_csv("/content/spam.csv")
```

Explore the dataset

```
dataset.info()
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5572 entries, 0 to 5571
Data columns (total 2 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Category    5572 non-null   object
1   Message     5572 non-null   object
dtypes: object(2)
memory usage: 87.2+ KB
```

```
dataset.info
```

Output:

Splitting our data into X and y.

```
<bound method DataFrame.info of      Category
0      ham  Go until jurong point, crazy.. Available only ...
1      ham                Ok lar... Joking wif u oni...
2     spam  Free entry in 2 a wkly comp to win FA Cup fina...
3      ham  U dun say so early hor... U c already then say...
4      ham  Nah I don't think he goes to usf, he lives aro...
...
5567    spam  This is the 2nd time we have tried 2 contact u...
5568     ham                Will ü b going to esplanade fr home?
5569     ham  Pity, * was in mood for that. So...any other s...
5570     ham  The guy did some bitching but I acted like i'd...
5571     ham                Rofl. Its true to its name

[5572 rows x 2 columns]>
```

```
X = dataset['Category'].values
```

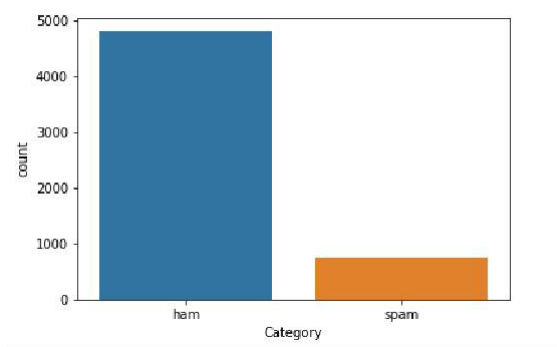
```
y = dataset['Message'].values
```

```
sns.countplot(x='Category',data=dataset)
```

```
plt. savefig("exercise3a.png")
```

```
files. download("exercise3a.png")
```

Output:



Splitting our data into training and testing.

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.2, random_state=0)
```

```
# Converting String to Integer
```

Converting text into integer using CountVectorizer()

```
cv = CountVectorizer()
```

```
X_train = cv.fit_transform(X_train)
```

```
X_test = cv.transform(X_test)
```

Applying SVM algorithm

```
from sklearn.svm
```

```
import SVC
```

```
classifier = SVC(kernel = 'rbf', random_state = 0)
```

```
classifier.fit(X_train, y_train)
```

Output:

```
SVC(random_state=0)
```

```
print(classifier.score(X_test,y_test))
```

Output:

```
0.003587443946188341
```

RESULT:

Thus the Spam mail detection using Support vector machine was implemented successfully.

Ex. No. : 3 Implement facial recognition application with artificial neural network.

PROBLEM STATEMENT:

Write a program to implement facial recognition application with artificial neural network

AIM:

To implement facial recognition application with artificial neural network

PROCEDURE:

Step 1: Import the data

Step 2: Transform the data

Step 3: Construct the tensor

Step 4: Build the model

Step 5: Train and evaluate the model Step 6: Improve the model

PROGRAM:

Import the data

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import mglearn
from sklearn.datasets
import fetch_lfw_people
from sklearn.decomposition
import PCA
people = fetch_lfw_people(min_faces_per_person=20, resize=0.7)
print(people.data.shape)
print(people.target.shape)
image_shape = people.images[0].shape
fig, axes = plt.subplots(2, 5, figsize=(15, 8), subplot_kw={'xticks': (), 'yticks': ()})
for target, image, ax in zip(people.target, people.images, axes.ravel()):
    ax.imshow(image)
    ax.set_title(people.target_names[target])
```

Output:

```
(3023, 5655)
(3023,)
```



!pip install mglearn

```
Looking in indexes: https://pypi.org/simple, https://us-python.pkg.dev/colab-wheels/public/simple/
Collecting mglearn
  Downloading mglearn-0.1.9.tar.gz (540 kB)
    540 kB 14.8 MB/s
Requirement already satisfied: numpy in /usr/local/lib/python3.7/dist-packages (from mglearn) (1.21.6)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.7/dist-packages (from mglearn) (3.2.2)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.7/dist-packages (from mglearn) (1.0.2)
Requirement already satisfied: pandas in /usr/local/lib/python3.7/dist-packages (from mglearn) (1.3.5)
Requirement already satisfied: pillow in /usr/local/lib/python3.7/dist-packages (from mglearn) (7.1.2)
Requirement already satisfied: cycler in /usr/local/lib/python3.7/dist-packages (from mglearn) (0.11.0)
Requirement already satisfied: imageio in /usr/local/lib/python3.7/dist-packages (from mglearn) (2.4.1)
Requirement already satisfied: joblib in /usr/local/lib/python3.7/dist-packages (from mglearn) (1.1.0)
Requirement already satisfied: python-dateutil>=2.1 in /usr/local/lib/python3.7/dist-packages (from matplotlib->mglearn) (2.8.2)
Requirement already satisfied: kiwisolver>=1.0.1 in /usr/local/lib/python3.7/dist-packages (from matplotlib->mglearn) (1.4.4)
Requirement already satisfied: pyparsing!=2.0.4,!=2.1.2,!=2.1.6,>=2.0.1 in /usr/local/lib/python3.7/dist-packages (from matplotlib->mglearn) (3.0.9)
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.7/dist-packages (from kiwisolver->=1.0.1->matplotlib->mglearn) (4.1.1)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.7/dist-packages (from python-dateutil->=2.1->matplotlib->mglearn) (1.15.0)
Requirement already satisfied: pytz>=2017.3 in /usr/local/lib/python3.7/dist-packages (from pandas->mglearn) (2022.1)
Requirement already satisfied: scipy>=1.1.0 in /usr/local/lib/python3.7/dist-packages (from scikit-learn->mglearn) (1.7.3)
Requirement already satisfied: threadpoolctl>=2.0.0 in /usr/local/lib/python3.7/dist-packages (from scikit-learn->mglearn) (3.1.0)
Building wheels for collected packages: mglearn
  Building wheel for mglearn (setup.py) ... done
  Created wheel for mglearn: filename=mglearn-0.1.9-py3-none-any.whl size=582639 sha256=fb84a9b286cdd563bfb25c05929a68e5ba40fac593cfcdb39c352ca94
  Stored in directory: /root/.cache/pip/wheels/f1/17/e1/1728d6dcd70187b6b6c3750cb3508798f2b1d57c9d3214b08b
Successfully built mglearn
Installing collected packages: mglearn
```

```
import numpy as np
```

```
import tensorflow as tfnp.random.seed(180)
```

```
from sklearn.model_selection
```

```
import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(people.data, people.target, test_size=0.2,
random_state=20)
```

```
y_train = y_train.astype(int)y_test = y_test.astype(int) batch_size =len(X_train)
```

```
print(X_train.shape, y_train.shape,y_test.shape )
```

Output:

```
(2418, 5655) (2418,) (605,)
```

```
print("people.images.shape: {}".format(people.images.shape)) print("Number of classes:
{}".format(len(people.target_names)))
```

Output:

```
people.images.shape: (3023, 87, 65)
```

```
Number of classes: 62
```

```
counts = np.bincount(people.target)
```

```
# print counts next to target names
```

```
for i, (count, name) in enumerate(zip(counts, people.target_names)):print("{0:25}
{1:3}".format(name, count), end='')
```

```
if (i + 1) % 3 == 0:
```

```
    print()
```

Output:

Alejandro Toledo	39	Alvaro Uribe	35	Amelie Mauresmo	21
Andre Agassi	36	Angelina Jolie	20	Ariel Sharon	77
Arnold Schwarzenegger	42	Atal Bihari Vajpayee	24	Bill Clinton	29
Carlos Menem	21	Colin Powell	236	David Beckham	31
Donald Rumsfeld	121	George Robertson	22	George W Bush	530
Gerhard Schroeder	109	Gloria Macapagal Arroyo	44	Gray Davis	26
Guillermo Coria	30	Hamid Karzai	22	Hans Blix	39
Hugo Chavez	71	Igor Ivanov	20	Jack Straw	28
Jacques Chirac	52	Jean Chretien	55	Jennifer Aniston	21
Jennifer Capriati	42	Jennifer Lopez	21	Jeremy Greenstock	24
Jiang Zemin	20	John Ashcroft	53	John Negroponte	31
Jose Maria Aznar	23	Juan Carlos Ferrero	28	Junichiro Koizumi	60
Kofi Annan	32	Laura Bush	41	Lindsay Davenport	22
Lleyton Hewitt	41	Luiz Inacio Lula da Silva	48	Mahmoud Abbas	29
Megawati Sukarnoputri	33	Michael Bloomberg	20	Naomi Watts	22
Nestor Kirchner	37	Paul Bremer	20	Pete Sampras	22
Recep Tayyip Erdogan	30	Ricardo Lagos	27	Roh Moo-hyun	32
Rudolph Giuliani	26	Saddam Hussein	23	Serena Williams	52
Silvio Berlusconi	33	Tiger Woods	23	Tom Daschle	25
Tom Ridge	33	Tony Blair	144	Vicente Fox	32
Vladimir Putin	49	Winona Ryder	24		

Transform the data

```
mask = np.zeros(people.target.shape, dtype=np.bool)
```

```
for target in np.unique(people.target):
```

```
mask[np.where(people.target == target)[0][:50]] = 1
```

```
X_people = people.data[mask]
```

```
y_people = people.target[mask]
```

```
# scale the grayscale values to be  
between 0 and 1# instead of 0 and 255
```

```
for better numeric stability X_people =  
X_people / 255.
```

Output:

```
/usr/local/lib/python3.7/dist-packages/ipykernel_launcher.py:1: DeprecationWarning: `np.bool` is a deprecated alias for the builtin `bool`. To silence  
Deprecation in NumPy 1.20; for more details and guidance: https://numpy.org/devdocs/release/1.20.0-notes.html#deprecations  
"""Entry point for launching an IPython kernel.
```

```
from sklearn.neighbors import KNeighborsClassifier
```

```
# split the data into training and test sets
```

```
X_train, X_test, y_train, y_test = train_test_split( X_people,  
y_people, stratify=y_people, random_state=0) # build a
```

```
KNeighborsClassifier using one neighbor
```

```
knn = KNeighborsClassifier(n_neighbors=1)
```

```
knn.fit(X_train, y_train)
```

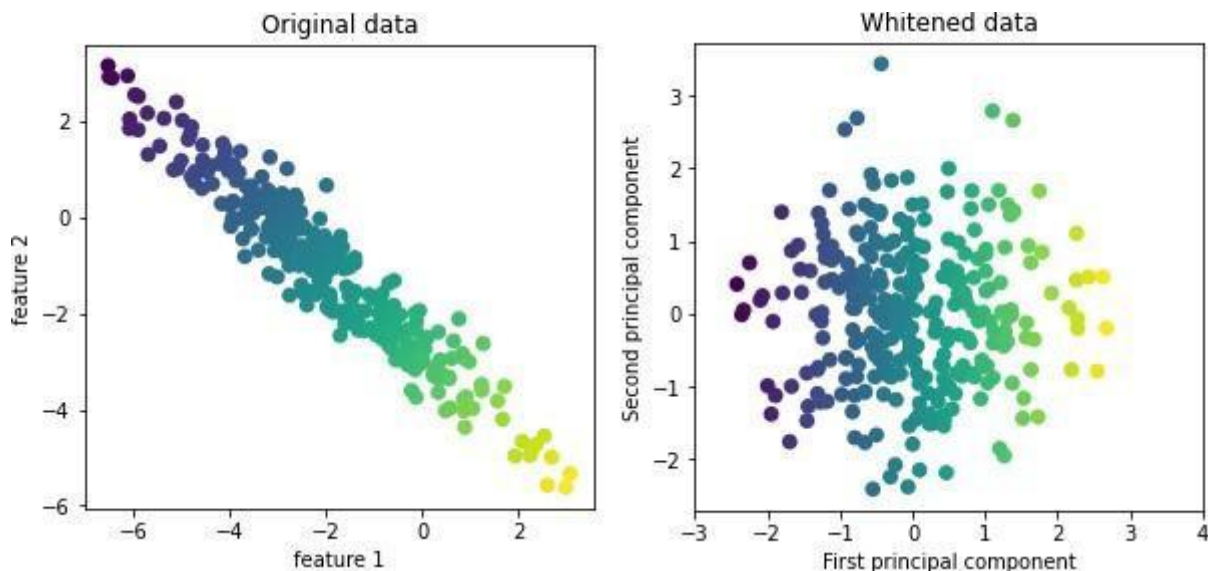
```
print("Test set score of 1-nn: {:.2f}".format(knn.score(X_test, y_test)))
```

Output:

```
Test set score of 1-nn: 0.23
```

```
mglearn.plots.plot_pca_whitening()
```

Output:



```
pca = PCA(n_components=100, whiten=True,
random_state=0).fit(X_train)X_train_pca =
pca.transform(X_train)
X_test_pca = pca.transform(X_test)
print("X_train_pca.shape:
{}".format(X_train_pca.shape))
```

```
X_train_pca.shape: (1547, 100)
knn =
KNeighborsClassifier(n_neighbor
s=1)knn.fit(X_train_pca, y_train)
```

```
print("Test set accuracy: {:.2f}".format(knn.score(X_test_pca, y_test)))
```

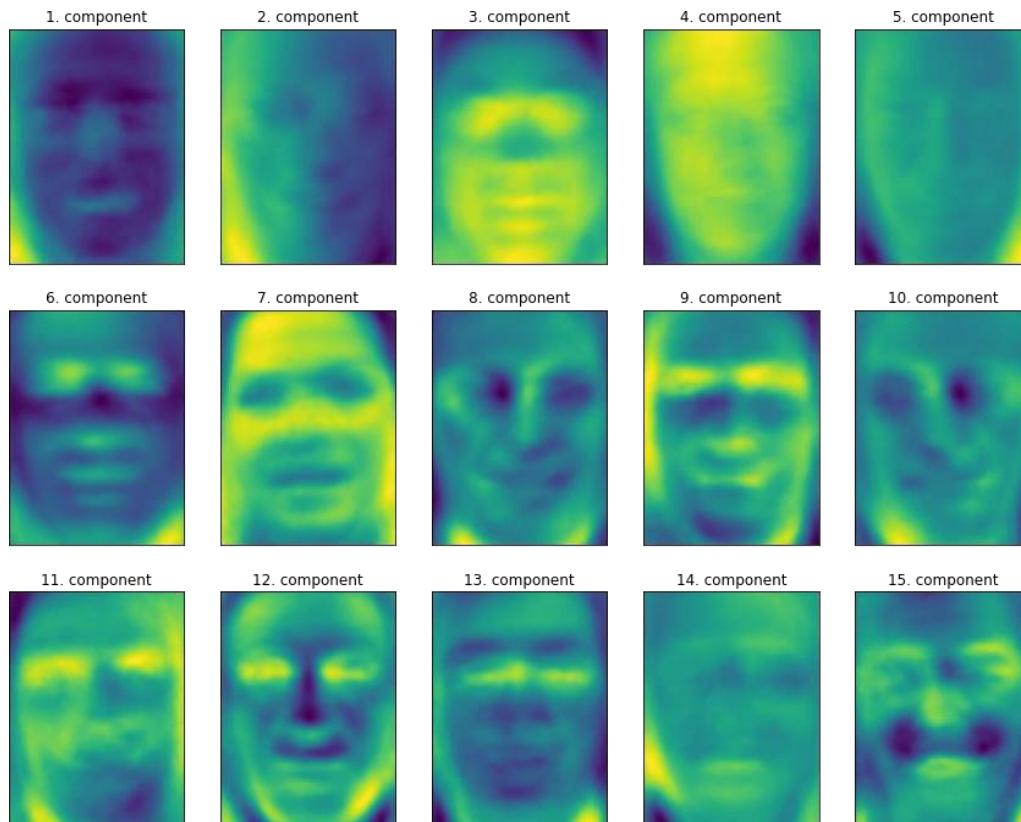
Output:

Test set accuracy: 0.31

```
print("pca.components_.shape: {}".format(pca.components_.shape))
pca.components_.shape: (100, 5655)
```

```
fix, axes = plt.subplots(3, 5, figsize=(15, 12),subplot_kw={'xticks': (),
'yticks': ()})
for i, (component, ax) in enumerate(zip(pca.components_, axes.ravel())):
    ax.imshow(component.reshape(image_shape),
    cmap='viridis')
    ax.set_title("{} component".format((i + 1)))
```

Output:



```
# resclae
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
# Train
X_train_scaled = scaler.fit_transform(X_train.astype(int))# test
X_test_scaled = scaler.fit_transform(X_test.astype(int))
feature_columns = [tf.feature_column.numeric_column('x', shape=X_train_scaled.shape[1:])]

```

Construct the tensor

```
import numpy as np
import tensorflow as tf
tf.random.seed(180)

```

Build the model

```
estimator =
    tf.estimator.DNNClassifier(feature_columns=feature_col
        umns, hidden_units=[300, 100], n_classes=10,
        model_dir = '/train/DNN')

```

Train and evaluate the model

```
train_input = tf.compat.v1.estimator.inputs.numpy_input_fn(x={"x": X_train_scaled},
    y=y_train, batch_size=50,
    shuffle=False,
    num_epochs=None)

```

Improve the model

```
estimator.train(input_fn = train_input, steps=1000)
eval_input = tf.estimator.inputs.numpy_input_fn(
    x={"x": X_test_scaled},

```

```
y=y_test,  
shuffle=False, batch_size=X_test_scaled.shape[0],num_epochs=0)  
estimator.evaluate(eval_input,steps=None)
```

RESULT:

Thus the facial recognition application with artificial neural network was implemented successfully.

Ex. No. : 4**Study and implement amazon toolkit: Sagemaker.****PROBLEM STATEMENT:**

Use Sage maker to create new notebooks to work the following:

Upload data, train and tune models, move back and forth between steps to adjust models, compareresults, and deploy models.

AIM:

To Study and implementation of amazon toolkit: Sagemaker.

PROCEDURE:

Step 1: Create a SageMaker notebook instance

Step 2: Prepare the data

Step 3: Train the model to learn from the data

Step 4: Deploy the model

Step 5: Evaluate your ML model's performance

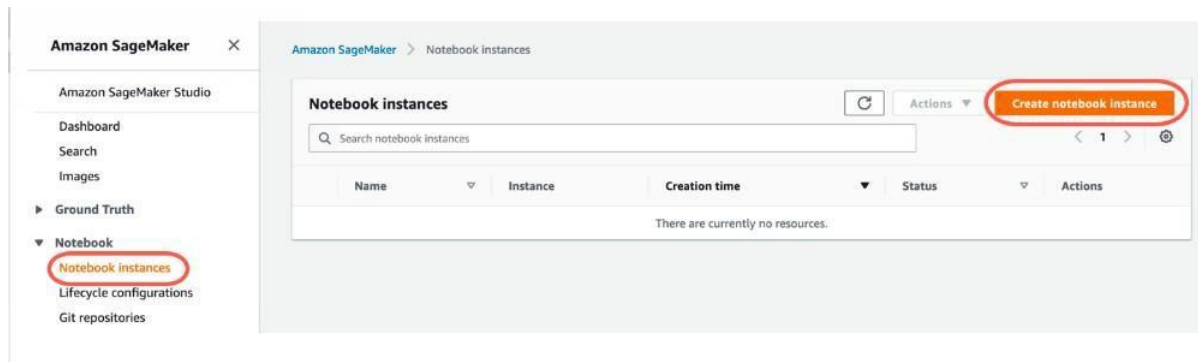
PROGRAM:**Create an Amazon SageMaker notebook instance for data preparation**

In this step, you create the notebook instance that you use to download and process your data. As part of the creation process, you also create an Identity and Access Management (IAM) role that allows Amazon SageMaker to access data in Amazon S3.

- a. Sign in to the Amazon SageMaker console, and in the top right corner, select your preferred AWS Region. This tutorial uses the US West (Oregon) Region.



- b. In the left navigation pane, choose Notebook instances, then choose Create notebookinstance.



c. following fields:

- For Notebook instance name, type *SageMaker-Tutorial*.
- For Notebook instance type, choose *ml.t2.medium*.
- For Elastic inference, keep the default selection of *none*.

Create notebook instance

Amazon SageMaker provides pre-built fully managed notebook instances that run Jupyter notebooks. The notebook instances include example code for common model training and hosting exercises. [Learn more](#)

Notebook instance settings

Notebook instance name

Maximum of 63 alphanumeric characters. Can include hyphens (-), but not spaces. Must be unique within your account in an AWS Region.

Notebook instance type

Elastic Inference [Learn more](#)

► **Additional configuration**

- d. In the Permissions and encryption section, for IAM role, choose Create a new role, and in the Create an IAM role dialog box, select Any S3 bucket and choose Create role.

Note: If you already have a bucket that you'd like to use instead, choose Specific S3 buckets and specify the bucket name.

Create an IAM role

Passing an IAM role gives Amazon SageMaker permission to perform actions in other AWS services on your behalf. Creating a role here will grant permissions described by the [AmazonSageMakerFullAccess](#) IAM policy to the role you create.

The IAM role you create will provide access to:

- ☒ **S3 buckets you specify - optional**
 - ☒ **Any S3 bucket**
Allow users that have access to your notebook instance access to any bucket and its contents in your account.
 - ☐ **Specific S3 buckets**

Comma delimited. ARNs, "" and "/" are not supported.
 - ☐ **None**
- ☒ **Any S3 bucket with "sagemaker" in the name**
- ☒ **Any S3 object with "sagemaker" in the name**
- ☒ **Any S3 object with the tag "sagemaker" and value "true"** [See Object tagging](#)
- ☒ **S3 bucket with a Bucket Policy allowing access to SageMaker** [See S3 bucket policies](#)

Amazon SageMaker creates the AmazonSageMaker-ExecutionRole-*** role.

Permissions and encryption

IAM role
Notebook instances require permissions to call other services including SageMaker and S3. Choose a role or let us create a role with the [AmazonSageMakerFullAccess](#) IAM policy attached.

☒ **Success! You created an IAM role.**
[AmazonSageMaker-ExecutionRole-20201204T135178](#)

Root access - optional

- ☒ **Enable - Give users root access to the notebook**
- ☐ **Disable - Don't give users root access to the notebook**
Lifecycle configurations always have root access

Encryption key - optional
Encrypt your notebook data. Choose an existing KMS key or enter a key's ARN.

e. Keep the default settings for the remaining options and choose Create notebook instance.

In the Notebook instances section, the new SageMaker-Tutorial notebook instance is displayed with a Status of Pending. The notebook is ready when the Status changes to InService.

Amazon SageMaker > Notebook instances

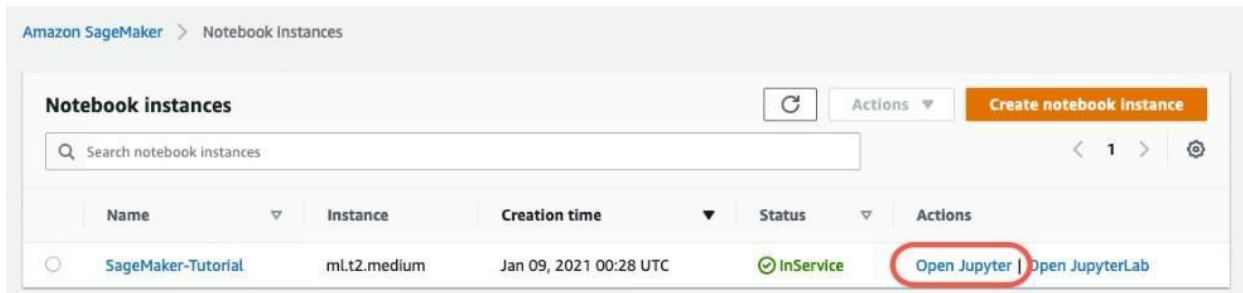
Notebook instances

	Name	Instance	Creation time	Status	Actions
☐	SageMaker-Tutorial	ml.t2.medium	Jan 09, 2021 00:28 UTC	☒ Pending	-

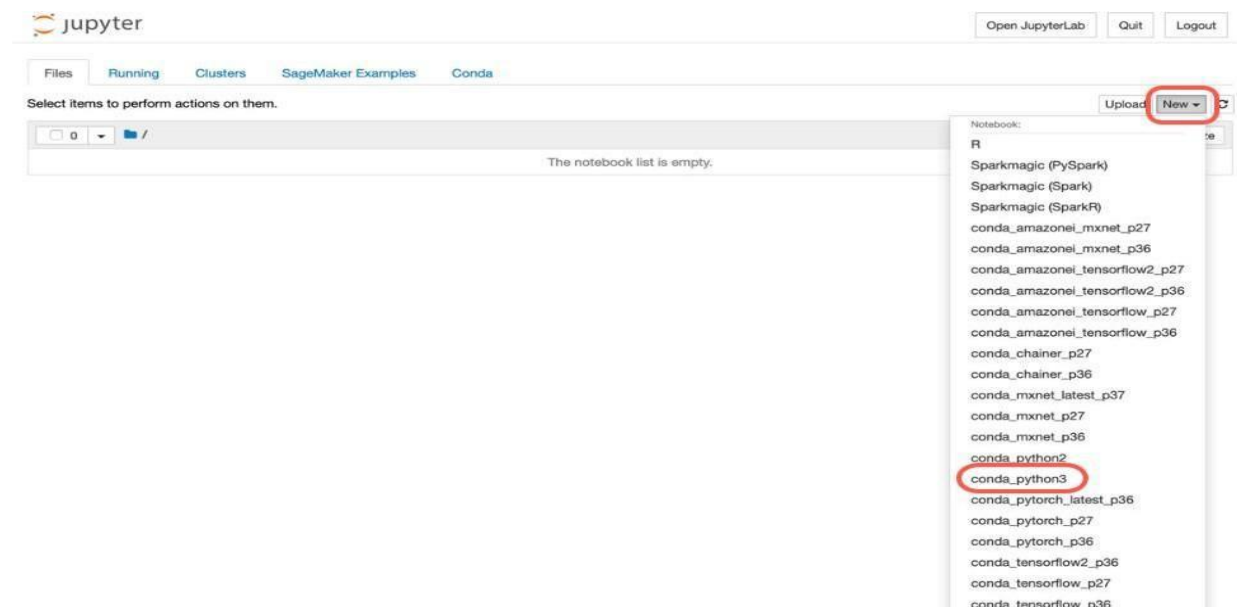
Prepare the data

In this step, you use your Amazon SageMaker notebook instance to preprocess the data that you need to train your machine learning model and then upload the data to Amazon S3.

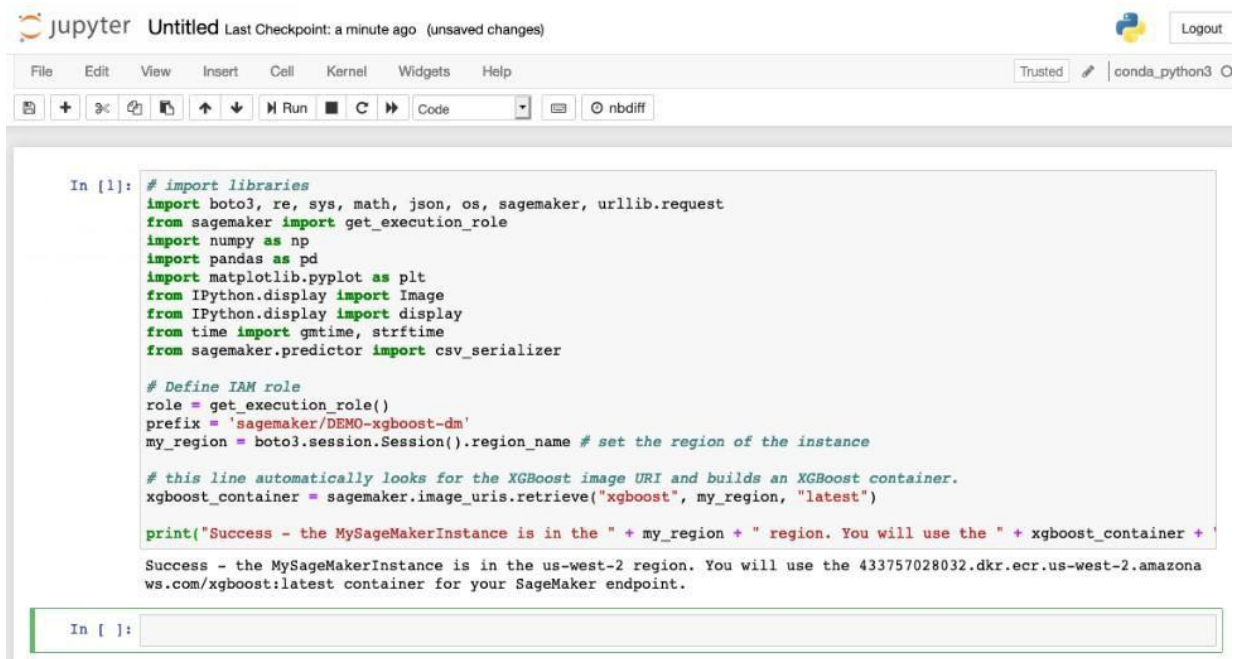
- a. After your SageMaker-Tutorial notebook instance status changes to InService, choose OpenJupyter.



- b. In Jupyter, choose New and then choose conda_python3.



- c. In a new code cell on your Jupyter notebook, copy and paste the following code and choose Run.



```

In [1]: # import libraries
import boto3, re, sys, math, json, os, sagemaker, urllib.request
from sagemaker import get_execution_role
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import Image
from IPython.display import display
from time import gmtime, strftime
from sagemaker.predictor import csv_serializer

# Define IAM role
role = get_execution_role()
prefix = 'sagemaker/DEMO-xgboost-dm'
my_region = boto3.session.Session().region_name # set the region of the instance

# this line automatically looks for the XGBoost image URI and builds an XGBoost container.
xgboost_container = sagemaker.image_uris.retrieve("xgboost", my_region, "latest")

print("Success - the MySageMakerInstance is in the " + my_region + " region. You will use the " + xgboost_container + " container for your SageMaker endpoint.")

Success - the MySageMakerInstance is in the us-west-2 region. You will use the 433757028032.dkr.ecr.us-west-2.amazonaws.com/xgboost:latest container for your SageMaker endpoint.

```

This code imports the required libraries and defines the environment variables you need to prepare the data, train the ML model, and deploy the ML model.

```

# import libraries
import boto3, re, sys, math, json, os, sagemaker,
urllib.request
from sagemaker import get_execution_role
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import Image
from IPython.display import display
from time import gmtime, strftime
from sagemaker.predictor import csv_serializer

# Define IAM role
role = get_execution_role()
prefix = 'sagemaker/DEMO-xgboost-dm'
my_region = boto3.session.Session().region_name # set the region of the instance

# this line automatically looks for the XGBoost image URI and builds an XGBoost
container.
xgboost_container = sagemaker.image_uris.retrieve("xgboost", my_region,
"latest")

print("Success - the MySageMakerInstance is in the " + my_region + " region. You will use the "
+ xgboost_container + " container for your SageMaker endpoint.")

```

d. Create the S3 bucket to store your data. Copy and paste the following code into the next code cell and choose Run.

Note: Make sure to replace the bucket_name your-s3-bucket-name with a unique S3 bucket name. If you don't receive a success message after running the code, change the bucket name and try again.

```

bucket_name = 'your-s3-bucket-name' # <--- CHANGE THIS VARIABLE TO A
UNIQUE NAME FOR YOUR BUCKET

```

```
s3 =
boto3.resource('s3')
try:
    if my_region == 'us-east-1':
        s3.create_bucket(Bucket=bucket_name)
    else:
        s3.create_bucket(Bucket=bucket_name,
CreateBucketConfiguration={ 'LocationConstraint':my_region })
    print('S3 bucket created
successfully')except Exception as e:
    print('S3 error: ',e)
```

e. Download the data to your SageMaker instance and load the data into a dataframe. Copy and paste the following code into the next code cell and choose Run.

```
try:
    urllib.request.urlretrieve ("https://d1.awsstatic.com/tmt/build-train-deploy-machine-
learning-model-
sagemaker/bank_clean.27f01fbbdf43271788427f3682996ae29ceca05d.csv",
"bank_clean.csv")
    print('Success: downloaded
bank_clean.csv.')except Exception as e:
    print('Data load error: ',e)
```

```
try:
    model_data = pd.read_csv('./bank_clean.csv',index_col=0)
    print('Success: Data loaded into
dataframe.')except Exception as e:
    print('Data load error: ',e)
```

f. Shuffle and split the data into training data and test data. Copy and paste the following code into the next code cell and choose Run.

The training data (70% of customers) is used during the model training loop. You use gradient- based optimization to iteratively refine the model parameters. Gradient-based optimization is a way to find model parameter values that minimize the model error, using the gradient of the model loss function.

The test data (remaining 30% of customers) is used to evaluate the performance of the model and measure how well the trained model generalizes to unseen data.

```
train_data, test_data = np.split(model_data.sample(frac=1, random_state=1729), [int(0.7 *
len(model_data))])
print(train_data.shape, test_data.shape)
```

Train the ML model

In this step, you use your training dataset to train your machine learning model.

a. In a new code cell on your Jupyter notebook, copy and paste the following code and choose Run.

This code reformats the header and first column of the training data and then loads the data from the S3 bucket. This step is required to use the Amazon SageMaker pre-built XGBoost algorithm.

```
pd.concat([train_data['y_yes'], train_data.drop(['y_no', 'y_yes'], axis=1)],
axis=1).to_csv('train.csv', index=False, header=False)
boto3.Session().resource('s3').Bucket(bucket_name).Object(os.path.join(prefix,
'train/train.csv')).upload_file('train.csv')
s3_input_train = sagemaker.inputs.TrainingInput(s3_data='s3://{}/{}/train'.format(bucket_name,
prefix), content_type='csv')
```

b. Set up the Amazon SageMaker session, create an instance of the XGBoost model (an estimator), and define the model's hyperparameters. Copy and paste the following code into the next code cell and choose Run.

```
sess = sagemaker.Session()

xgb = sagemaker.estimator.Estimator(xgboost_container, role, instance_count=1,
instance_type='ml.m4.xlarge', output_path='s3://{}/{}/output'.format(bucket_name,
prefix), sagemaker_session=sess)
xgb.set_hyperparameters(max_depth=5, eta=0.2, gamma=4, min_child_weight=6, subsample=0.8,
silent=0, objective='binary:logistic', num_round=100)
```

c. Start the training job. Copy and paste the following code into the next code cell and choose Run.

This code trains the model using gradient optimization on a ml.m4.xlarge instance. After a few minutes, you should see the training logs being generated in your Jupyter notebook.

```
xgb.fit({'train': s3_input_train})
```

Deploy the model

In this step, you deploy the trained model to an endpoint, reformat and load the CSV data, then run the model to create predictions.

a. In a new code cell on your Jupyter notebook, copy and paste the following code and choose Run.

This code deploys the model on a server and creates a SageMaker endpoint that you can access. This step may take a few minutes to complete.

```
xgb_predictor = xgb.deploy(initial_instance_count=1, instance_type='ml.m4.xlarge')
```

b. To predict whether customers in the test data enrolled for the bank product or not, copy the following code into the next code cell and choose Run.

```
from sagemaker.serializers import CSVSerializer
```

```
test_data_array = test_data.drop(['y_no', 'y_yes'], axis=1).values #load the data into an array
xgb_predictor.serializer = CSVSerializer() # set the serializer type
predictions = xgb_predictor.predict(test_data_array).decode('utf-8') # predict!
predictions_array = np.fromstring(predictions[1:], sep=',') # and turn the prediction into an array
print(predictions_array.shape)
```

Evaluate model performance

In this step, you evaluate the performance and accuracy of the machine learning model.

In a new code cell on your Jupyter notebook, copy and paste the following code and choose

Run. This code compares the actual vs. predicted values in a table called a confusion matrix.

Based on the prediction, we can conclude that you predicted a customer will enroll for a certificate of deposit accurately for 90% of customers in the test data, with a precision of 65% (278/429) for enrolled and 90% (10,785/11,928) for didn't enroll.

```
cm = pd.crosstab(index=test_data['y_yes'], columns=np.round(predictions_array),
rownames=['Observed'], colnames=['Predicted'])
tn = cm.iloc[0,0]; fn = cm.iloc[1,0]; tp = cm.iloc[1,1]; fp = cm.iloc[0,1];
p = (tp+tn)/(tp+tn+fp+fn)*100
print("\n{0:<20}{1:<4.1f}%\n".format("Overall Classification Rate: ", p))
print("{0:<15}{1:<15}{2:>8}".format("Predicted", "No Purchase",
"Purchase"))print("Observed")
print("{0:<15}{1:<2.0f}% ({2:<}){3:>6.0f}% ({4:<})".format("No
Purchase",tn/(tn+fn)*100,tn, fp/(tp+fp)*100, fp))
print("{0:<16}{1:<1.0f}% ({2:<}){3:>7.0f}% ({4:<}) \n".format("Purchase",
fn/(tn+fn)*100,fn,tp/(tp+fp)*100, tp))
```

Clean up

In this step, you terminate the resources you used in this lab.

Important: Terminating resources that are not actively being used reduces costs and is a best practice. Not terminating your resources will result in charges to your account.

a. Delete your endpoint: In your Jupyter notebook, copy and paste the following code and choose Run.

```
xgb_predictor.delete_endpoint(delete_endpoint_config=True)
```

Delete your training artifacts and S3 bucket: In your Jupyter notebook, copy and paste the following code and choose Run.

```
bucket_to_delete = boto3.resource('s3').Bucket(bucket_name)
```

```
bucket_to_delete.objects.all().delete()
```

c. Delete your SageMaker Notebook: Stop and delete your SageMaker Notebook.

1. Open the [SageMaker console](#).
2. Under Notebooks, choose Notebook instances.
3. Choose the notebook instance that you created for this tutorial, then choose Actions, Stop. The notebook instance takes up to several minutes to stop. When Status changes to Stopped, move onto the next step.
4. Choose Actions, then Delete.
5. Choose Delete.

RESULT:

Thus the Sage Maker was studied and implemented successfully.

Ex. No. : 5**Implement k-nearest neighbours classification.****PROBLEM STATEMENT:**

Write a program to implement k-nearest neighbor algorithm to classify the iris data set. Print both correct and wrong predictions.

AIM:

To implement a program for Classification of Iris dataset using K-nearest Neighbour algorithm.

PROCEDURE:

Step 1: Importing Libraries

Step 2: Importing Dataset

Step 3: Split Dataset

Step 4: Training Model

Step 5: Running Predictions Step 6: Checking Validation

PROGRAM:**Importing Libraries**

```
from sklearn.model_selection import
train_test_split
from sklearn.neighbors
import KNeighborsClassifier from
sklearn.metrics import
classification_report from sklearn.metrics
import confusion_matrix
import pandas as pd
import numpy as np
from sklearn import datasets
```

Importing Dataset

```
iris=datasets.load_iris()
iris_data=iris.data
iris_labels=iris.target
print(iris_data)
```

Output:

```
[[5.1 3.5 1.4 0.2]
 [4.9 3.  1.4 0.2]
 [4.7 3.2 1.3 0.2]
 [4.6 3.1 1.5 0.2]
 [5.  3.6 1.4 0.2]
 [5.4 3.9 1.7 0.4]
 [4.6 3.4 1.4 0.3]
 [5.  3.4 1.5 0.2]
 [4.4 2.9 1.4 0.2]
 [4.9 3.1 1.5 0.1]

 [5.4 3.7 1.5 0.2]
 [4.8 3.4 1.6 0.2]
 [4.8 3.  1.4 0.1]
```

```

[4.3 3.  1.1 0.1]
[5.8 4.  1.2 0.2]
[5.7 4.4 1.5 0.4]
[5.4 3.9 1.3 0.4]
[5.1 3.5 1.4 0.3]
[5.7 3.8 1.7 0.3]
[5.1 3.8 1.5 0.3]
[5.4 3.4 1.7 0.2]
[5.1 3.7 1.5 0.4]
[4.6 3.6 1.  0.2]
[5.1 3.3 1.7 0.5]
[4.8 3.4 1.9 0.2]
[5.  3.  1.6 0.2]
[5.  3.4 1.6 0.4]
[5.2 3.5 1.5 0.2]
[5.2 3.4 1.4 0.2]
[4.7 3.2 1.6 0.2]
[4.8 3.1 1.6 0.2]
[5.4 3.4 1.5 0.4]
[5.2 4.1 1.5 0.1]
[5.5 4.2 1.4 0.2]
[4.9 3.1 1.5 0.2]
[5.  3.2 1.2 0.2]
[5.5 3.5 1.3 0.2]
[4.9 3.6 1.4 0.1]
[4.4 3.  1.3 0.2]
[5.1 3.4 1.5 0.2]
[5.  3.5 1.3 0.3]
[4.5 2.3 1.3 0.3]
[4.4 3.2 1.3 0.2]
[5.  3.5 1.6 0.6]
[5.1 3.8 1.9 0.4]
[4.8 3.  1.4 0.3]
[5.1 3.8 1.6 0.2]
[4.6 3.2 1.4 0.2]
[5.3 3.7 1.5 0.2]
[5.  3.3 1.4 0.2]
[7.  3.2 4.7 1.4]
[6.4 3.2 4.5 1.5]
[6.9 3.1 4.9 1.5]
[5.5 2.3 4.  1.3]
[6.5 2.8 4.6 1.5]
[5.7 2.8 4.5 1.3]
[6.3 3.3 4.7 1.6]
[4.9 2.4 3.3 1. ]
[6.6 2.9 4.6 1.3]
[5.2 2.7 3.9 1.4]
[5.  2.  3.5 1. ]
[5.9 3.  4.2 1.5]
[6.  2.2 4.  1. ]
[6.1 2.9 4.7 1.4]
[5.6 2.9 3.6 1.3]
[6.7 3.1 4.4 1.4]
[5.6 3.  4.5 1.5]
[5.8 2.7 4.1 1. ]
[6.2 2.2 4.5 1.5]
[5.6 2.5 3.9 1.1]
[5.9 3.2 4.8 1.8]
[6.1 2.8 4.  1.3]
[6.3 2.5 4.9 1.5]
[6.1 2.8 4.7 1.2]
[6.4 2.9 4.3 1.3]

```

[6.6 3. 4.4 1.4]
 [6.8 2.8 4.8 1.4]
 [6.7 3. 5. 1.7]
 [6. 2.9 4.5 1.5]
 [5.7 2.6 3.5 1.]
 [5.5 2.4 3.8 1.1]
 [5.5 2.4 3.7 1.]
 [5.8 2.7 3.9 1.2]
 [6. 2.7 5.1 1.6]
 [5.4 3. 4.5 1.5]
 [6. 3.4 4.5 1.6]
 [6.7 3.1 4.7 1.5]
 [6.3 2.3 4.4 1.3]
 [5.6 3. 4.1 1.3]
 [5.5 2.5 4. 1.3]
 [5.5 2.6 4.4 1.2]
 [6.1 3. 4.6 1.4]
 [5.8 2.6 4. 1.2]
 [5. 2.3 3.3 1.]
 [5.6 2.7 4.2 1.3]
 [5.7 3. 4.2 1.2]
 [5.7 2.9 4.2 1.3]
 [6.2 2.9 4.3 1.3]
 [5.1 2.5 3. 1.1]
 [5.7 2.8 4.1 1.3]
 [6.3 3.3 6. 2.5]
 [5.8 2.7 5.1 1.9]
 [7.1 3. 5.9 2.1]
 [6.3 2.9 5.6 1.8]
 [6.5 3. 5.8 2.2]
 [7.6 3. 6.6 2.1]
 [4.9 2.5 4.5 1.7]
 [7.3 2.9 6.3 1.8]
 [6.7 2.5 5.8 1.8]
 [7.2 3.6 6.1 2.5]
 [6.5 3.2 5.1 2.]
 [6.4 2.7 5.3 1.9]
 [6.8 3. 5.5 2.1]
 [5.7 2.5 5. 2.]
 [5.8 2.8 5.1 2.4]
 [6.4 3.2 5.3 2.3]
 [6.5 3. 5.5 1.8]
 [7.7 3.8 6.7 2.2]
 [7.7 2.6 6.9 2.3]
 [6. 2.2 5. 1.5]
 [6.9 3.2 5.7 2.3]
 [5.6 2.8 4.9 2.]
 [7.7 2.8 6.7 2.]
 [6.3 2.7 4.9 1.8]
 [6.7 3.3 5.7 2.1]
 [7.2 3.2 6. 1.8]
 [6.2 2.8 4.8 1.8]
 [6.1 3. 4.9 1.8]
 [6.4 2.8 5.6 2.1]
 [7.2 3. 5.8 1.6]
 [7.4 2.8 6.1 1.9]
 [7.9 3.8 6.4 2.]
 [6.4 2.8 5.6 2.2]
 [6.3 2.8 5.1 1.5]
 [6.1 2.6 5.6 1.4]
 [7.7 3. 6.1 2.3]
 [6.3 3.4 5.6 2.4]

```
[6.4 3.1 5.5 1.8]
[6.  3.  4.8 1.8]
[6.9 3.1 5.4 2.1]
[6.7 3.1 5.6 2.4]
[6.9 3.1 5.1 2.3]
[5.8 2.7 5.1 1.9]
[6.8 3.2 5.9 2.3]
[6.7 3.3 5.7 2.5]
[6.7 3.  5.2 2.3]
[6.3 2.5 5.  1.9]
[6.5 3.  5.2 2. ]
[6.2 3.4 5.4 2.3]
[5.9 3.  5.1 1.8]]
```

Split Dataset

```
x_train, x_test, y_train, y_test=(train_test_split(iris_data, iris_labels, test_size=0.20))
```

Training Model

```
classifier=KNeighborsClassifier(n_neighbors=6)
classifier.fit(x_train, y_train)
y_pred=classifier.predict(x_test)
```

Running Predictions

```
print("accuracy is")
print(classification_report(y_test, y_pred))
```

Output:

```
accuracy is
precision    recall  f1-score   support

      0       1.00      1.00      1.00        15
      1       1.00      1.00      1.00         8
      2       1.00      1.00      1.00         7

 accuracy
macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```

```
cm=confusion_matrix(y_test,
y_pred)cm
```

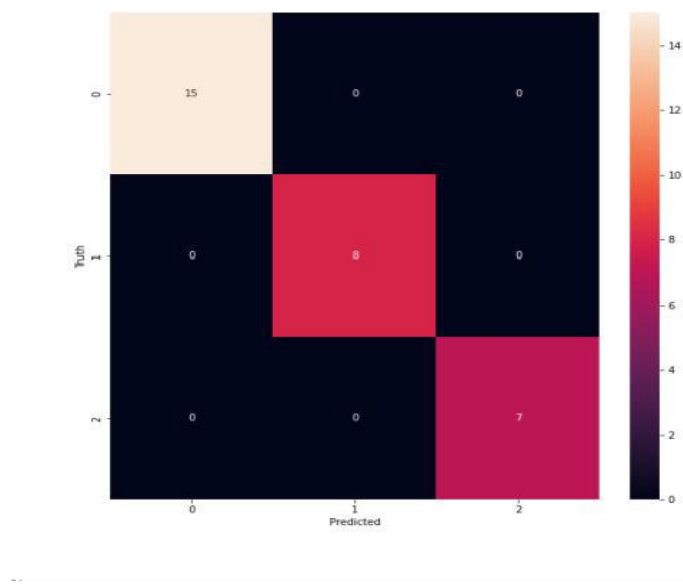
Output:

```
array([[15,  0,  0],
       [ 0,  8,  0],
       [ 0,  0,  7]])
```

Checking Validation

```
import matplotlib.pyplot as plt
import seaborn as sn
from google.colab import files
plt.figure(figsize=(10,10))
sn.heatmap(cm, annot=True)
plt.xlabel('Predicted')
plt.ylabel('Truth')
plt.savefig("exercise6.png")
files.download("exercise6.png")
```

Output:

**RESULT:**

Thus the Classification of Iris dataset using K-nearest Neighbour algorithm was implemented successfully.

Ex. No. : 6 Implement the non-parametric Locally Weighted Regression algorithm in order to fit data points.

PROBLEM STATEMENT:

Implement the non-parametric Locally Weighted Regression algorithm in Python in order to fit data points. Select the appropriate data set for your experiment and draw graphs.

AIM:

To implement non-parametric Locally Weighted Regression algorithm for data-point fitting.

PROCEDURE:

Step 1: Importing Libraries

Step 2: Importing Dataset

Step 3: Function to calculate weight matrix

Step 4: Visualize the data

PROGRAM:

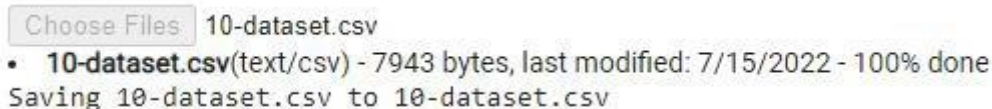
Importing Libraries

```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
```

Importing Dataset

```
from google.colab import filesupload = files.upload()
```

Output:



Choose Files 10-dataset.csv

- 10-dataset.csv(text/csv) - 7943 bytes, last modified: 7/15/2022 - 100% done

Saving 10-dataset.csv to 10-dataset.csv

```
data = pd.read_csv('10-dataset.csv')
bill = np.array(data.total_bill)
tip = np.array(data.tip)
```

Function to calculate weight matrix

```
def kernel(point, xmat, k):
    m,n = np.shape(xmat)
    weights = np.mat(np.eye((m)))
    for j in range(m): diff = point - X[j]
        weights[j,j] = np.exp(diff*diff.T/(-
            2.0*k**2))return weights

def localWeight(point, xmat, ymat, k):
    wei = kernel(point,xmat,k)
```

$$W = (X.T*(wei*X)).I*(X.T*(wei*y.mat.T))$$

```
def localWeightRegression(xmat, ymat, k):
    m, n = np.shape(xmat)
    ypred = np.zeros(m)
    for i in range(m):
        ypred[i] = xmat[i]*localWeight(xmat[i], xmat, ymat, k)
    return ypred

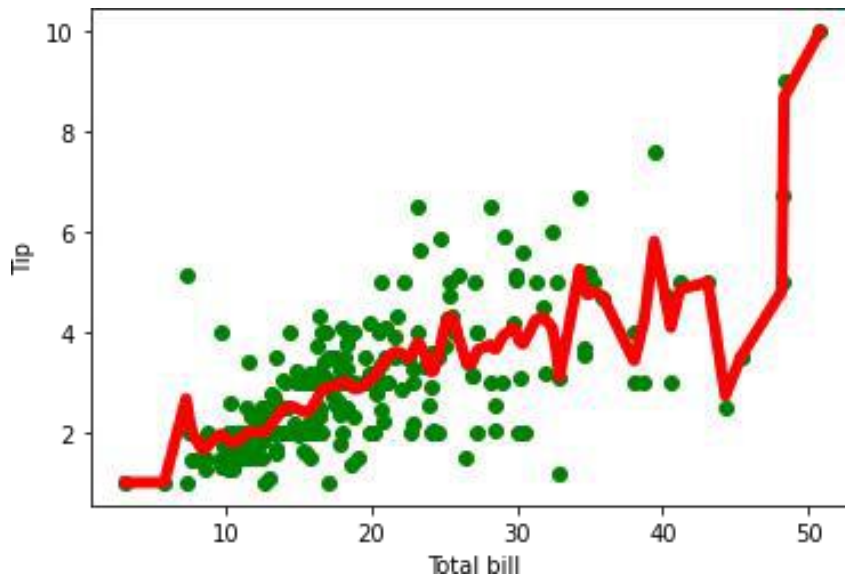
mbill = np.mat(bill)
mtip = np.mat(tip)

m = np.shape(mbill)[1]
one = np.mat(np.ones(m))
X = np.hstack((one.T, mbill.T))
ypred = localWeightRegression(X, mtip, 0.5)
SortIndex = X[:, 1].argsort(0)
xsort = X[SortIndex][:, 0]
```

Visualise the data

```
fig = plt.figure()
ax = fig.add_subplot(1, 1, 1)
ax.scatter(bill, tip, color='green')
ax.plot(xsort[:, 1], ypred[SortIndex], color='red', linewidth=5)
plt.xlabel("Total bill")
plt.ylabel("Tip")
plt.show()
plt.savefig("exercise8.png")
files.download("exercise8.png")
```

Output:



RESULT:

Thus the non-parametric Locally Weighted Regression algorithm for data-point fitting was implemented successfully.

Ex. No. : 7 Implement sentiment analysis using random forest optimization algorithm.

PROBLEM STATEMENT:

Write a program to implement Sentiment analysis using random forest optimization algorithm.

AIM:

To implement Sentiment analysis using random forest optimization algorithm.

PROCEDURE:

Step 1: Import the required libraries

Step 2: Load Restaurant Reviews dataset

Step 3: Explore the dataset

Step 4: Data Preprocessing

Step 5: Build the Model

Step 6: Train the Model Step 7: Running Predictions

PROGRAM:

Import the required libraries

```
import numpy as np
```

```
import pandas as pd
```

Load Restaurant Reviews dataset

```
from google.colab import filesupload = files.upload()
```

Output:

Choose Files Restaurant_Reviews.tsv
 • Restaurant_Reviews.tsv(n/a) - 61332 bytes, last modified: 9/25/2020 - 100% done
 Saving Restaurant_Reviews.tsv to Restaurant_Reviews.tsv

Explore the dataset

```
df=pd.read_csv('Restaurant_Reviews.tsv',delimiter='\t', quoting=3)
```

```
df.head()
```

Output:

	Review	Liked
0	Wow... Loved this place.	1
1	Crust is not good.	0
2	Not tasty and the texture was just nasty.	0
3	Stopped by during the late May bank holiday of...	1
4	The selection on the menu was great and so wer...	1

df.shape

Output:

(1000, 2)

Data Preprocessing

```
import nltk
import re
nltk.download('stopwords')
from nltk.corpus import stopwords
from nltk.stem.porter import PorterStemmer
```

Output:

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Unzipping corpora/stopwords.zip.
```

```
# Cleaning
the reviews
corpus = []
for i in range(0,1000):
    # Cleaning special character from the reviews
    review = re.sub(pattern='[^a-zA-Z]',repl='', string=df['Review'][i])
    # Converting the entire review into
    lower case
    review = review.lower()
    # Tokenizing the review by words
    review_words = review.split()
    # Removing the stop words
    review_words = [word for word in review_words if not word in
set(stopwords.words('english'))]
    # Stemming the words
    ps = PorterStemmer()
    review = [ps.stem(word) for word in review_words]
    # Joining the stemmed words
    review = ' '.join(review)
    # Creating a corpus
    corpus.append(review) corpus[0:10]
```

Output:

```
['wow love place',
'crust good',
'tasti textur nasti',
'stop late may bank holiday rick steve recommend love',
'select menu great price',
'get angri want damn pho',
'honeslti tast fresh',
'potato like rubber could tell made ahead time kept warmer',
'fri great',
'great touch']
```

Build the Model

```
# Creating the Bag of Words model
from sklearn.feature_extraction.text
import CountVectorizer
cv = CountVectorizer(max_features=1500)
X = cv.fit_transform(corpus).toarray() y = df.iloc[:, 1].values
```

Train the Model

```

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=0)
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier(n_estimators=35)
model.fit(X_train, y_train)
RandomForestClassifier(n_estimators=35)
model.score(X_test, y_test)

```

Output:

0.7

Running Predictions

```

# Predicting the Test set results
y_pred = model.predict(X_test) # Accuracy, Precision and Recall
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
score1 = accuracy_score(y_test, y_pred)
score2 = precision_score(y_test, y_pred)
score3 = recall_score(y_test, y_pred)
print("---- Scores ----")
print("Accuracy score is: {}".format(round(score1*100,2)))
print("Precision score is: {}".format(round(score2,2)))
print("Recall score is: {}".format(round(score3,2)))

```

Output:

```

---- Scores ----
Accuracy score is: 70.0%
Precision score is: 0.8
Recall score is: 0.55

```

Making the Confusion Matrix

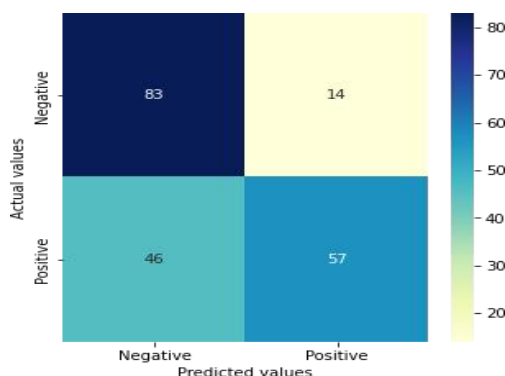
```

from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred)
# Plotting the confusion matrix
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline plt.figure(figsize=(5,5))
sns.heatmap(cm, annot=True, cmap="YlGnBu", xticklabels=['Negative', 'Positive'], yticklabels=['Negative', 'Positive'])
plt.xlabel('Predicted values')
plt.ylabel('Actual values')

```

Output:

Text(23.999999999999996, 0.5, 'Actual values')



```
# Hyperparameter tuning the Naive
Bayes Classifier
best_accuracy = 0.0
n_estimators = 5
for i in np.arange(1,30,2):
    temp_classifier = RandomForestClassifier(n_estimators=i)
    temp_classifier.fit(X_train, y_train)
    temp_y_pred = temp_classifier.predict(X_test)
    score = accuracy_score(y_test, temp_y_pred)
    print("Accuracy score for n_estimators={} is: {}".format(round(i,1),
    round(score*100,2)))
    if score>best_accuracy:
        best_accuracy = score
        n_estimators_val = i
print('.....')
print("The best accuracy is {}% with n_estimators value as {}".format(round(best_accuracy*100,
2), round(n_estimators_val,1)))
```

Output:

```
Accuracy score for n_estimators=1 is: 72.5%
Accuracy score for n_estimators=3 is: 64.0%
Accuracy score for n_estimators=5 is: 72.0%
Accuracy score for n_estimators=7 is: 68.5%
Accuracy score for n_estimators=9 is: 65.5%
Accuracy score for n_estimators=11 is: 70.5%
Accuracy score for n_estimators=13 is: 70.0%
Accuracy score for n_estimators=15 is: 70.0%
Accuracy score for n_estimators=17 is: 69.5%
Accuracy score for n_estimators=19 is: 70.0%
Accuracy score for n_estimators=21 is: 69.5%
Accuracy score for n_estimators=23 is: 73.0%
Accuracy score for n_estimators=25 is: 71.0%
Accuracy score for n_estimators=27 is: 67.0%
Accuracy score for n_estimators=29 is: 70.5%
-----
The best accuracy is 73.0% with n_estimators value as 23
```

```
model = RandomForestClassifier(n_estimators=n_estimators_val)
model.fit(X_train, y_train)
RandomForestClassifier(n_estimators=19)
```

```
def predict_sentiment(sample_review):
    sample_review = re.sub(pattern='[^a-zA-Z]', repl='', string =sample_review)
    sample_review = sample_review.lower()
    sample_review_words = sample_review.split()
    sample_review_words = [word for word in sample_review_words if not word in
    set(stopwords.words('english'))]
    ps = PorterStemmer()
    final_review = [ps.stem(word) for word in sample_review_words]
    final_review = ' '.join(final_review)
    temp = cv.transform([final_review]).toarray()
    return model.predict(temp)

sample_review = 'The food is really good here.'
if predict_sentiment(sample_review):
    print("This is a POSITIVE review.")
```

```
else:  
    print("This is a NEGATIVE review!")  
Output:  
    This is a POSITIVE review.
```

```
# Predicting values  
sample_review = 'Food was pretty bad and the service was  
very slow.'  
if predict_sentiment(sample_review):  
    print("This is a POSITIVE review.")  
else:  
    print("This is a NEGATIVE review!")
```

Output:

```
This is a NEGATIVE review!
```

```
# Predicting values  
sample_review = 'The food was absolutely wonderful and nicely presentation'  
if predict_sentiment(sample_review):  
    print("This is a POSITIVE review.")  
else:  
    print("This is a NEGATIVE review!")
```

Output:

```
This is a POSITIVE review.
```

RESULT:

Thus the Sentiment analysis using random forest optimization algorithm was implemented successfully.

Ex. No. : 8 Write a program to construct a Bayesian network considering medical data**PROBLEM STATEMENT:**

Write a program to Construct Bayesian network model for medical diagnosis.

AIM:

To Construct Bayesian network model for medical diagnosis.

PROCEDURE:

Step 1: Import the required libraries

Step 2: Load the heart dataset

Step 3: Explore the dataset

Step 4: Build the Model

PROGRAM:**Import the required libraries**

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Load the heart dataset

```
from google.colab
import filesupload = files.upload()
```

Output:

Choose Files heart.csv

- heart.csv(text/csv) - 11328 bytes, last modified: 7/1/2021 - 100% done

Saving heart.csv to heart.csv

Explore the dataset

```
df=pd.read_csv('heart.csv')df.head()
```

Output:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	63	1	3	145	233	1	0	150	0	2.3	0	0	1	1
1	37	1	2	130	250	0	1	187	0	3.5	0	0	2	1
2	41	0	1	130	204	0	0	172	0	1.4	2	0	2	1
3	56	1	1	120	236	0	1	178	0	0.8	2	0	2	1
4	57	0	0	120	354	0	1	163	1	0.6	2	0	2	1

```
df.shape
```

Output:

```
df.isnull().sum()
```

Output:

```

age      0
sex      0
cp       0
trestbps 0
chol     0
fbs      0
restecg  0
thalach  0
exang    0
oldpeak  0
slope    0
ca       0
thal     0
target   0
dtype: int64

```

```

df.drop(['oldpeak','slope','ca','thal'],axis=1,inplace=True)
df.head()

```

Output:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	target
0	63	1	3	145	233	1	0	150	0	1
1	37	1	2	130	250	0	1	187	0	1
2	41	0	1	130	204	0	0	172	0	1
3	56	1	1	120	236	0	1	178	0	1
4	57	0	0	120	354	0	1	163	1	1

```
df.info()
```

Output:

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 303 entries, 0 to 302
Data columns (total 10 columns):
#   Column      Non-Null Count  Dtype
---  ------      -
0   age         303 non-null    int64
1   sex         303 non-null    int64
2   cp          303 non-null    int64
3   trestbps    303 non-null    int64
4   chol        303 non-null    int64
5   fbs         303 non-null    int64
6   restecg     303 non-null    int64
7   thalach     303 non-null    int64
8   exang       303 non-null    int64
9   target      303 non-null    int64
dtypes: int64(10)
memory usage: 23.8 KB

```

```
df.describe()
```

Output:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	target
count	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000
mean	54.366337	0.683168	0.966997	131.623762	246.264026	0.148515	0.528053	149.646865	0.326733	0.544554
std	9.082101	0.466011	1.032052	17.538143	51.830751	0.356198	0.525860	22.905161	0.469794	0.498835
min	29.000000	0.000000	0.000000	94.000000	126.000000	0.000000	0.000000	71.000000	0.000000	0.000000
25%	47.500000	0.000000	0.000000	120.000000	211.000000	0.000000	0.000000	133.500000	0.000000	0.000000
50%	55.000000	1.000000	1.000000	130.000000	240.000000	0.000000	1.000000	153.000000	0.000000	1.000000
75%	61.000000	1.000000	2.000000	140.000000	274.500000	0.000000	1.000000	166.000000	1.000000	1.000000
max	77.000000	1.000000	3.000000	200.000000	564.000000	1.000000	2.000000	202.000000	1.000000	1.000000

!pip install pgmpy

```
Looking in indexes: https://pypi.org/simple, https://us-python.pkg.dev/colab-wheels/public/simple/
Collecting pgmpy
  Downloading pgmpy-0.1.19-py3-none-any.whl (1.9 MB)
    |#####| 1.9 MB 29.4 MB/s
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.7/dist-packages (from pgmpy) (1.0.2)
Requirement already satisfied: tqdm in /usr/local/lib/python3.7/dist-packages (from pgmpy) (4.64.0)
Requirement already satisfied: networkx in /usr/local/lib/python3.7/dist-packages (from pgmpy) (2.6.3)
Requirement already satisfied: pyparsing in /usr/local/lib/python3.7/dist-packages (from pgmpy) (3.0.9)
Requirement already satisfied: statsmodels in /usr/local/lib/python3.7/dist-packages (from pgmpy) (0.10.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.7/dist-packages (from pgmpy) (1.7.3)
Requirement already satisfied: numpy in /usr/local/lib/python3.7/dist-packages (from pgmpy) (1.21.6)
Requirement already satisfied: torch in /usr/local/lib/python3.7/dist-packages (from pgmpy) (1.12.0+cu113)
Requirement already satisfied: pandas in /usr/local/lib/python3.7/dist-packages (from pgmpy) (1.3.5)
Requirement already satisfied: joblib in /usr/local/lib/python3.7/dist-packages (from pgmpy) (1.1.0)
Requirement already satisfied: python-dateutil>=2.7.3 in /usr/local/lib/python3.7/dist-packages (from pandas->pgmpy) (2.8.2)
Requirement already satisfied: pytz>=2017.3 in /usr/local/lib/python3.7/dist-packages (from pandas->pgmpy) (2022.1)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.7/dist-packages (from python-dateutil>=2.7.3->pandas->pgmpy) (1.15.0)
Requirement already satisfied: threadpoolctl>=2.0.0 in /usr/local/lib/python3.7/dist-packages (from scikit-learn->pgmpy) (3.1.0)
Requirement already satisfied: patsy>=0.4.0 in /usr/local/lib/python3.7/dist-packages (from statsmodels->pgmpy) (0.5.2)
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.7/dist-packages (from torch->pgmpy) (4.1.1)
Installing collected packages: pgmpy
Successfully installed pgmpy-0.1.19
```

Build the Model

```
from pgmpy.models import BayesianModel
```

```
from pgmpy.estimators import MaximumLikelihoodEstimator, BayesianEstimator
```

Output:

```
/usr/local/lib/python3.7/dist-packages/statsmodels/tools/_testing.py:19: FutureWarning: pandas.util.testing is deprecated. Use the functions in the p
import pandas.util.testing as tm
```

```
df.columns
```

Output:

```
Index(['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach',
      'exang', 'target'],
      dtype='object')
```

```
model=BayesianModel([(('age','target'),('sex','target'),('exang','target'),('cp','target'),('target','restecg'),
                        ('target','chol')])
```

```
model.fit(df, estimator=MaximumLikelihoodEstimator)
```

```
print(model.get_cpds('age'))
```

Output:

```
+-----+
| age(29) | 0.00330033 |
+-----+
| age(34) | 0.00660066 |
+-----+
| age(35) | 0.0132013 |
+-----+
| age(37) | 0.00660066 |
+-----+
| age(38) | 0.00990099 |
+-----+
| age(39) | 0.0132013 |
+-----+
| age(40) | 0.00990099 |
+-----+
| age(41) | 0.0330033 |
+-----+
| age(42) | 0.0264026 |
+-----+
| age(43) | 0.0264026 |
```

```
/usr/local/lib/python3.7/dist-packages/pgmpy/models/BayesianModel.py:10:
```

```
FutureWarning: BayesianModel has been renamed to BayesianNetwork. Please use
BayesianNetwork class, BayesianModel will be removed in future.
```

```
FutureWarning,
```

```
print(model.get_cpds('sex'))
```

Output:

```
+-----+
| sex(0) | 0.316832 |
+-----+
| sex(1) | 0.683168 |
+-----+
```

```
print(model.get_cpds('exang'))
```

Output:

```
+-----+
| exang(0) | 0.673267 |
+-----+
| exang(1) | 0.326733 |
+-----+
```

```
print(model.get_cpds('chol'))
```

Output:

```
+-----+-----+-----+-----+
| target | target(0) | target(1) |
+-----+-----+-----+-----+
| chol(126) | 0.0 | 0.006060606060606061 |
+-----+-----+-----+-----+
| chol(131) | 0.007246376811594203 | 0.0 |
+-----+-----+-----+-----+
| chol(141) | 0.0 | 0.006060606060606061 |
+-----+-----+-----+-----+
| chol(149) | 0.007246376811594203 | 0.006060606060606061 |
+-----+-----+-----+-----+
| chol(157) | 0.0 | 0.006060606060606061 |
+-----+-----+-----+-----+
| chol(160) | 0.0 | 0.006060606060606061 |
+-----+-----+-----+-----+
| chol(164) | 0.007246376811594203 | 0.0 |
+-----+-----+-----+-----+
| chol(166) | 0.007246376811594203 | 0.0 |
+-----+-----+-----+-----+
| chol(167) | 0.007246376811594203 | 0.0 |
+-----+-----+-----+-----+
```

```
| chol(168) | 0.0 | 0.006060606060606061 |
+-----+-----+-----+
| chol(169) | 0.007246376811594203 | 0.0 |
+-----+-----+-----+
| chol(172) | 0.007246376811594203 | 0.0 |
+-----+-----+-----+
```

RESULT:

Thus the Construction of Bayesian network model for medical diagnosis was implemented successfully.

Ex. No. : 9**Implement online fraud detection.****PROBLEM STATEMENT:**

Write a program to implement online fraud detection using machine learning algorithm.

AIM:

To implement online fraud detection using machine learning algorithm.

PROCEDURE:

Step 1: Import the required libraries

Step 2: Load the heart dataset

Step 3: Explore the dataset

Step 4: Perform Isolation Forest Algorithm and Local Outlier Factor(LOF) Algorithm to dataset.

PROGRAM:**Import the required libraries**

```
import numpy as np
import pandas as pd
import sklearn
import scipy
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import classification_report, accuracy_score
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import LocalOutlierFactor
from sklearn.svm import OneClassSVM
from pylab import rcParams
rcParams['figure.figsize'] = 14, 8
RANDOM_SEED = 42
LABELS = ["Normal", "Fraud"]
```

Load the heart dataset

```
from google.colab import filesupload = files.upload()
```

Output:

Choose Files creditcard.csv

- creditcard.csv(text/csv) - 150828752 bytes, last modified: 3/2/2021 - 100% done

Saving creditcard.csv to creditcard.csv

Explore the dataset

```
data = pd.read_csv('creditcard.csv', sep=',')
data.head()
```

Output:

	Time	V1	V2	V3	V4	V5	V6	V7	V8	V9	...	V21	V22	V23	V24	V2
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	0.098698	0.363787	...	-0.018307	0.277838	-0.110474	0.066928	0.12853
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	0.085102	-0.255425	...	-0.225775	-0.638672	0.101288	-0.339846	0.16717
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	0.247676	-1.514654	...	0.247998	0.771679	0.909412	-0.689281	-0.32764
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609	0.377436	-1.387024	...	-0.108300	0.005274	-0.190321	-1.175575	0.64737
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941	-0.270533	0.817739	...	-0.009431	0.798278	-0.137458	0.141267	-0.20601

5 rows x 31 columns

```
data.info()
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 284807 entries, 0 to 284806
Data columns (total 31 columns):
 #   Column  Non-Null Count  Dtype
---  -
 0   Time    284807 non-null  float64
 1   V1       284807 non-null  float64
 2   V2       284807 non-null  float64
 3   V3       284807 non-null  float64
 4   V4       284807 non-null  float64
 5   V5       284807 non-null  float64
 6   V6       284807 non-null  float64
 7   V7       284807 non-null  float64
 8   V8       284807 non-null  float64
 9   V9       284807 non-null  float64
10  V10      284807 non-null  float64
11  V11      284807 non-null  float64
12  V12      284807 non-null  float64
13  V13      284807 non-null  float64
14  V14      284807 non-null  float64
15  V15      284807 non-null  float64
16  V16      284807 non-null  float64
17  V17      284807 non-null  float64
18  V18      284807 non-null  float64
19  V19      284807 non-null  float64
20  V20      284807 non-null  float64
```

```
data.isnull().values.any()
```

Output:

```
False
```

```
count_classes = pd.value_counts(data['Class'], sort = True)
```

```
count_classes.plot(kind = 'bar', rot=0)
```

```
plt.title("Transaction Class Distribution")
```

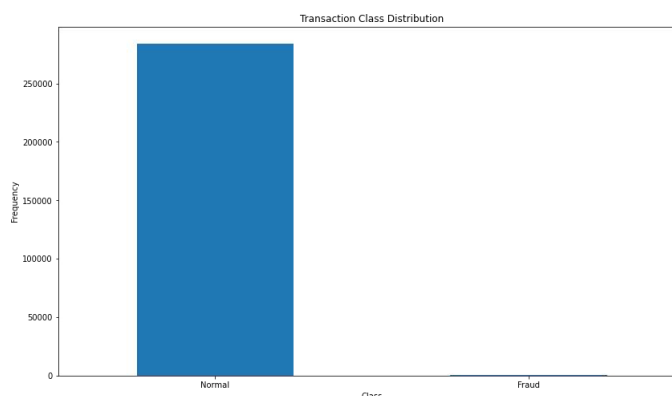
```
plt.xticks(range(2), LABELS)
```

```
plt.xlabel("Class")
```

```
plt.ylabel("Frequency")
```

Output:

```
Text(0, 0.5, 'Frequency')
```



```

fraud = data[data['Class']==1]
normal = data[data['Class']==0]
print(fraud.shape,normal.shape)

```

Output:

```
(492, 31) (284315, 31)
```

```
fraud.Amount.describe()
```

Output:

```

count      492.000000
mean       122.211321
std        256.683288
min         0.000000
25%         1.000000
50%         9.250000
75%        105.890000
max       2125.870000
Name: Amount, dtype: float64

```

```
normal.Amount.describe()
```

Output:

```

count      284315.000000
mean         88.291022
std         250.105092
min          0.000000
25%          5.650000
50%         22.000000
75%         77.050000
max       25691.160000
Name: Amount, dtype: float64

```

```

f, (ax1, ax2) = plt.subplots(2, 1, sharex=True)
f.suptitle('Amount per transaction by class')

```

```
bins = 50
```

```
ax1.hist(fraud.Amount,
```

```
bins = bins)
```

```
ax1.set_title('Fraud')
```

```
ax2.hist(normal.Amount,
```

```
bins = bins)
```

```
ax2.set_title('Normal')
```

```
plt.xlabel('Amount ($)')
```

```

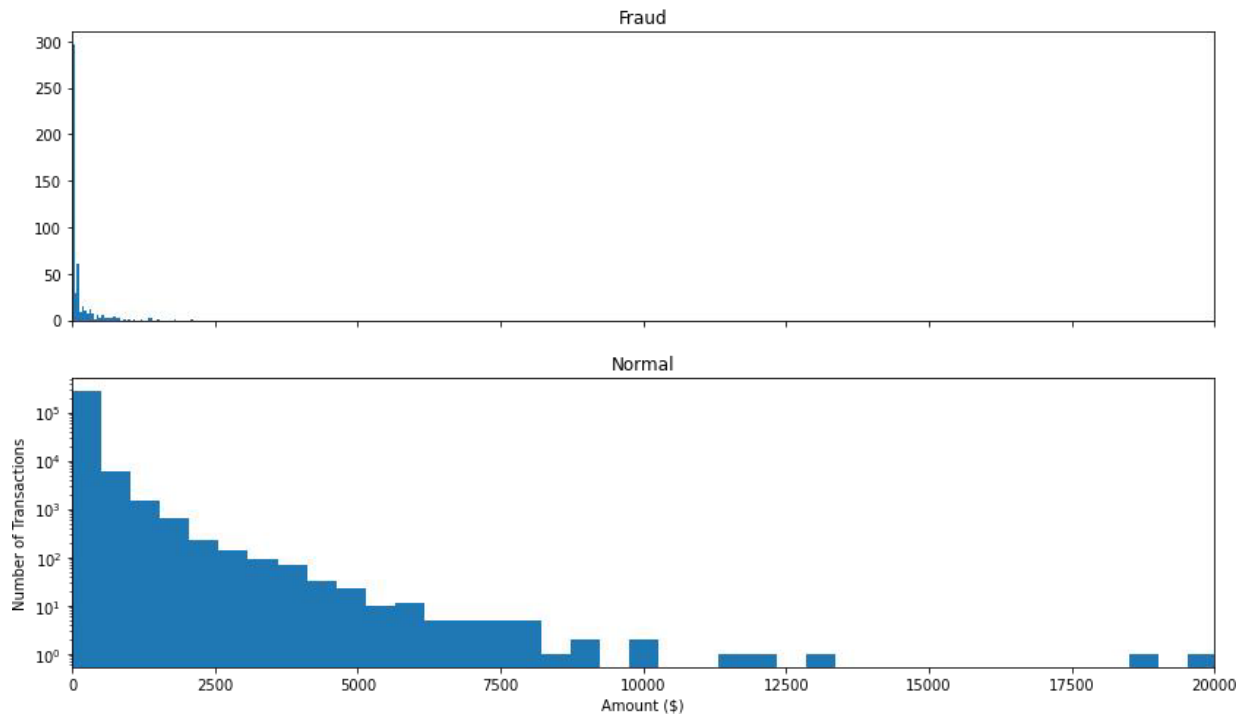
plt.ylabel('Number of
Transactions') plt.xlim((0,
20000))

```

```
plt.yscale('log')plt.show();
```

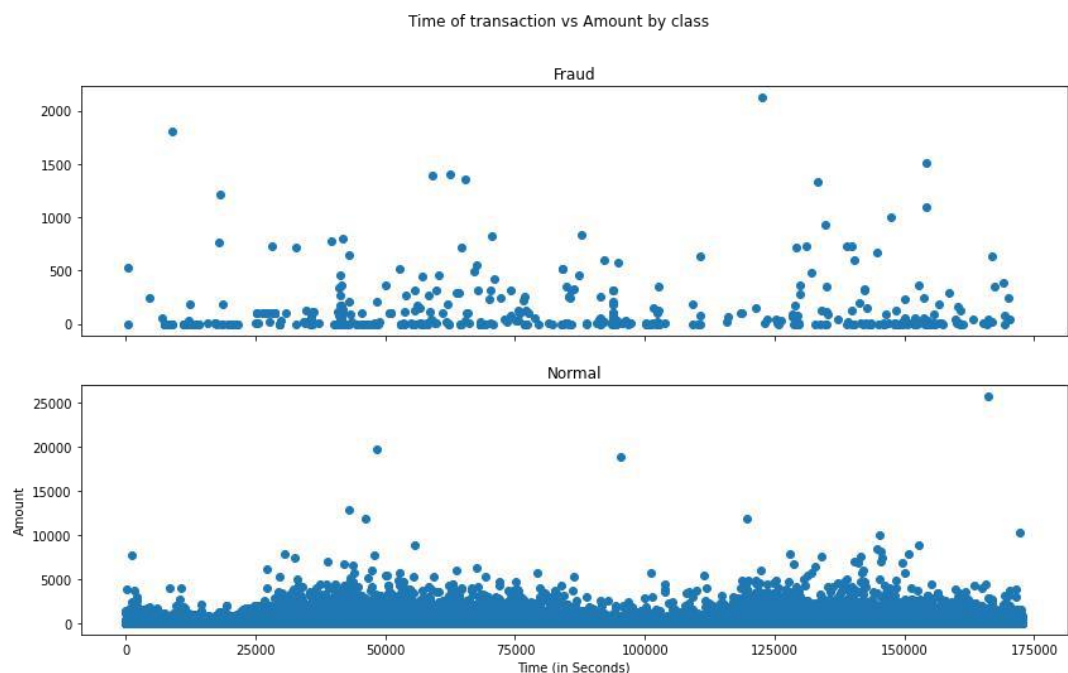
Output:

```
f, (ax1, ax2) = plt.subplots(2, 1, sharex=True)
Amount per transaction by class
```



```
f.suptitle('Time of transaction vs Amount by class')
ax1.scatter(fraud.Time, fraud.Amount) ax1.set_title('Fraud')
ax2.scatter(normal.Time, normal.Amount)
ax2.set_title('Normal')
plt.xlabel('Time (in Seconds)')
plt.ylabel('Amount')
plt.show()
```

Output:



```
data1= data.sample(frac = 0.1,random_state=1)data1.shape
```

Output:

```
(28481, 31)
```

```
data.shape
```

Output:

```
(284807, 31)
```

```
Fraud = data1[data1['Class']==1]
```

```
Valid = data1[data1['Class']==0]
```

```
outlier_fraction = len(Fraud)/float(len(Valid))
```

```
print(outlier_fraction)
```

```
print("Fraud Cases : {}".format(len(Fraud)))
```

```
print("Valid Cases : {}".format(len(Valid)))
```

Output:

```
0.0017234102419808666
```

```
Fraud
```

```
Cases : 49
```

```
Valid
```

```
Cases :
```

```
28432
```

Perform Isolation Forest Algorithm and Local Outlier Factor(LOF) Algorithm to dataset.

```
## Correlation
```

```
import seaborn as sns
```

```
#get correlations of each features in dataset
```

```
corrmat = data1.corr()
```

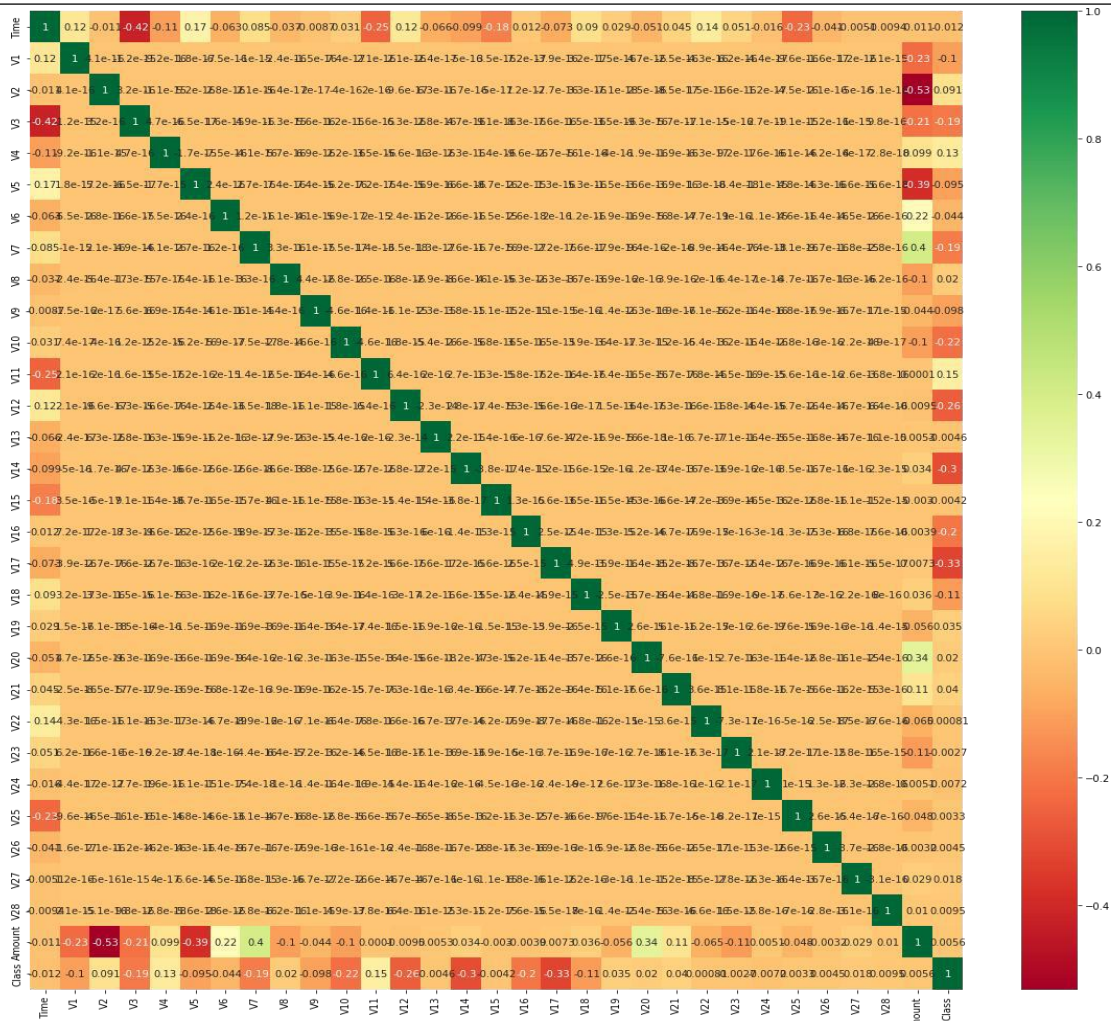
```
top_corr_features = corrmat.index
```

```
plt.figure(figsize=(20,20))
```

```
#plot heat map
```

```
g=sns.heatmap(data[top_corr_features].corr(),annot=True,cmap="RdYlGn")
```

Output:



#Create independent and Dependent Features

```
columns = data1.columns.tolist()
```

```
# Filter the columns to remove data we do not want
```

```
columns = [c for c in columns if c not in ["Class"]] # Store the variable we are predicting
```

```
target = "Class"
```

```
# Define a random state
```

```
state = np.random.RandomState(42)
```

```
X = data1[columns]
```

```
Y = data1[target]
```

```
X_outliers = state.uniform(low=0, high=1,
```

```
size=(X.shape[0],X.shape[1]))
```

```
# Print the
```

```
shapes of X & Y
```

```
print(X.shape)
```

```
print(Y.shape)
```

Output:

```
(28481, 30)
```

```
(28481, )
```

```
##Define the outlier detection methods
```

```
classifiers = {"Isolation Forest":IsolationForest(n_estimators=100,
```

```

max_samples=len(X),

contamination=outlier_fraction,random_state=state, verbose=0),
"Local Outlier Factor":LocalOutlierFactor(n_neighbors=20, algorithm='auto',
leaf_size=30, metric='minkowski',
p=2, metric_params=None, contamination=outlier_fraction)

}
type(classifiers)dict
n_outliers = len(Fraud)
for i, (clf_name,clf) in enumerate(classifiers.items()):
    #Fit the data and tag outliers
    if clf_name == "Local Outlier Factor":
        y_pred = clf.fit_predict(X)
        scores_prediction = clf.negative_outlier_factor_
        else:
            clf.fit(X)
            scores_prediction = clf.decision_function(X)

    y_pred = clf.predict(X)
    #Reshape the prediction values to 0 for Valid transactions , 1 for Fraud
    transactions
    y_pred[y_pred == 1] = 0
    y_pred[y_pred == -1] = 1
    n_errors = (y_pred != Y).sum()# Run Classification Metrics
    print("{}: {}".format(clf_name,n_errors))

    print("Accuracy Score :") print(accuracy_score(Y,y_pred))
    print("Classification Report :")
    print(classification_report(Y,y_pred))

```

Output:

```

/usr/local/lib/python3.7/dist-packages/sklearn/base.py:451:
  "X does not have valid feature names, but"
Isolation Forest: 73
Accuracy Score :
0.9974368877497279
Classification Report :

```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	28432
1	0.26	0.27	0.26	49
accuracy			1.00	28481
macro avg	0.63	0.63	0.63	28481
weighted avg	1.00	1.00	1.00	28481

```

Local Outlier Factor: 97
Accuracy Score :
0.9965942207085425
Classification Report :

```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	28432
1	0.02	0.02	0.02	49
accuracy			1.00	28481
macro avg	0.51	0.51	0.51	28481
weighted avg	1.00	1.00	1.00	28481

RESULT:

Thus the online fraud detection using machine learning algorithm was implemented successfully.

Ex. No. : 11

Implementation of different method os Defuzzification process

PROBLEM STATEMENT:

Write a program to implement different method os Defuzzification process

AIM:

To understand the concept of Defuzzification.

PROCEDURE:

There are namely three sets: Cold, Warm & Hot.

The following procedure is to be followed for the simulation:

Step 1: Set the range of each of the 3 sets mentioned above by adjusting the slider to the desired values and see the corresponding change in the graph.

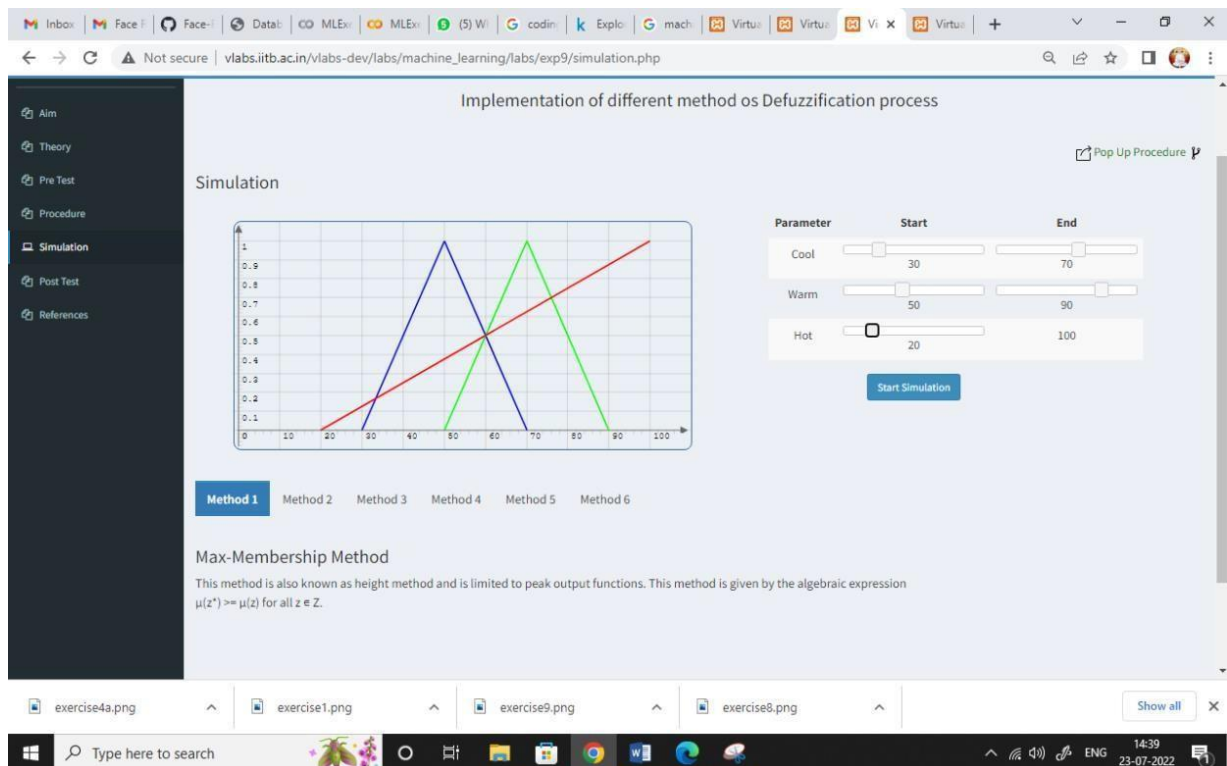
Step 2: Click on Start Simulation button.

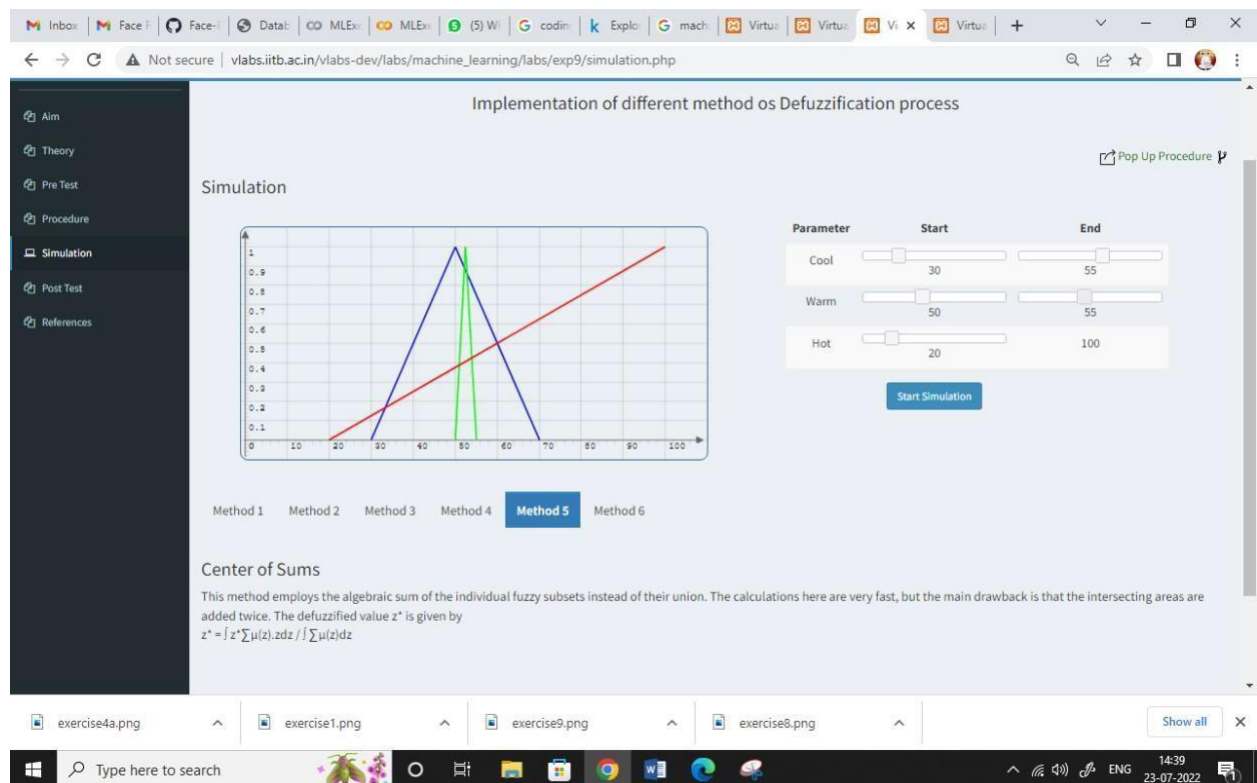
Step 3: Below the graph you can observe how the I/P is manipulated and the defuzzified value is calculated.

Step 4: You can switch between tabs to see the result for the each of the 6 methods of defuzzifications explained in the Theory section of the experiment.

Step 5: The explanation and the formula of the calculations is provided in the respective tabs for reference purpose.

After the entire simulation is completed, you may select other values for each set & I/P and then run the simulation.





Virtual lab link:

http://vlabs.iitb.ac.in/vlabs-dev/labs/machine_learning/labs/exp9/simulation.php

Ex. No. : 12**Understanding the concepts of Perceptron Learning Rule****PROBLEM STATEMENT:**

Write a program to understand the concepts of Perceptron Learning Rule

AIM:

To understand the perceptron learning rule that can be applied for supervised learning of neuralnetwork.

PROCEDURE:

The following procedure is to be followed for the simulation:

Step1: Change the learning rate if you want to by using the slider for the same.

Step2: Change the initial weights of the neural network if you want to by clicking on the links connecting the input nodes to summation nodes. You can see the weight changes in the alongside graph.

Step3: Once the simulation is started, these can't be changed..

Step4: After making changes as needed, click Start Simulation button. Now each input as highlighted in the graph will be chosen and learnt using the learning algorithm.

Step5: If you want to stop the simulation in between you can do so by clicking the Stop Simulation button.

Step6: After each input is completely processed, the Apply next I/P value button will get activated. Click on this button to learn using the next input.

Step7: As you will see the regions will converge towards correct classification after each iteration. For correct classification, the regions should be such that for each input, the output of only one neuron should be +1 and that of other neurons should be -1. Thus, in the graph, each point should fall under one region only. Initially, they will fall under multiple regions. But as you will see, the position of the regions changes after every iteration so that each point falls under one region only.

Step8: Thus you can consider the aim to be colours of the regions should match the colours of the points that they cover. And the regions should cover all the same colour points.

Simulation

→ The blue region in the graph depicts the boundary formed due to neuron 1, red region corresponds to neuron 2, and green region corresponds to neuron 3.

Learning Rate (η): 0.4

Here, the activation function used is **signum**:

$$\text{sgn}(x) = 1, \quad x \geq 0$$

$$= -1, \quad x < 0$$

To understand what calculations are happening check [this popup](#) and to understand the representations of vectors and matrices [click here](#) or hover your mouse over them.

Lab contributed by K. J. Somaiya College of Engineering

9. Predicting Stoc...docx exercise4a.png exercise1.png exercise9.png Show all

Type here to search

Simulation

→ Click on any line to change its weight. This will also update the regions in the graph.

→ You cannot change the parameters once you've started simulations.

→ The blue region in the graph depicts the boundary formed due to neuron 1, red region corresponds to neuron 2, and green region corresponds to neuron 3.

Learning Rate (η): 0.4

Here, the activation function used is **signum**:

$$\text{sgn}(x) = 1, \quad x \geq 0$$

$$= -1, \quad x < 0$$

To understand what calculations are happening check [this popup](#) and to understand the representations of vectors and matrices [click here](#) or hover your mouse over them.

9. Predicting Stoc...docx exercise4a.png exercise1.png exercise9.png Show all

Type here to search

Virtual lab link:

http://vlabs.iitb.ac.in/vlabs-dev/labs/machine_learning/labs/exp4/simulation.php

Ex. No. : 13**Implementation Back propagation Algorithm using ANN****PROBLEM STATEMENT:**

Build an Artificial Neural Network by implementing the Back propagation Algorithm and test the same using appropriate data sets.

AIM:

To Implement Back propagation Algorithm using ANN.

PROCEDURE:

Step 1: Perform Sigmoid Function

Step 2: Perform Derivative of Sigmoid Function

Step 3: Draw a random range of numbers uniformly of dim x*y

Step 4: Apply Back propagation

PROGRAM:

```
import numpy as np

X = np.array([[2, 9], [1, 5], [3, 6]], dtype=float)
y = np.array([92, 86, 89], dtype=float)
X = X/np.amax(X,axis=0) #maximum of X array
longitudinallyy = y/100
#Sigmoid Function
def sigmoid(x):
    return 1/(1 + np.exp(-x))

#Derivative of Sigmoid Function
def derivatives_sigmoid(x):
    return x * (1 - x)
#Variable initialization
epoch=5 #Setting training iterations
lr=0.1 #Setting learning rate

inputlayer_neurons = 2 #number of features in data set
hiddenlayer_neurons = 3 #number of hidden layers neurons
output_neurons = 1 #number of neurons at output layer #weight and bias
initialization

wh=np.random.uniform(size=(inputlayer_neurons,hiddenlayer_neurons))
bh=np.random.uniform(size=(1,hiddenlayer_neurons))
wout=np.random.uniform(size=(hiddenlayer_neurons,output_neurons))
bout=np.random.uniform(size=(1,output_neurons))

#draws a random range of numbers uniformly
of dim x*y
for i in range(epoch):
    #Forward Propagation
```

```

hinp1=np.dot(X,wh)
hinp=hinp1 + bh hlayer_act = sigmoid(hinp)
outinp1=np.dot(hlayer_act,wout)
outinp= outinp1+bout
output = sigmoid(outinp)

#Backpropagation
EO = y-output
outgrad = derivatives_sigmoid(output)
d_output = EO * outgrad
EH = d_output.dot(wout.T)
hiddengrad = derivatives_sigmoid(hlayer_act)#how much hidden layer wts contributed to error

d_hiddenlayer = EH * hiddengrad

wout += hlayer_act.T.dot(d_output) *lr # dotproduct of nextlayererror and
currentlayerop
wh += X.T.dot(d_hiddenlayer) *lr

print ("-----Epoch-", i+1, "Starts ----- ")
print("Input: \n" + str(X))
print("Actual Output: \n" + str(y))
print("Predicted Output: \n",output)
print ("-----Epoch-", i+1, "Ends ----- \n")

print("Input: \n" + str(X))
print("Actual Output: \n" + str(y))
print("Predicted Output: \n",output)

```

OUTPUT:

```

-----Epoch- 1 Starts -----
Input:
[[0.66666667 1.   ]
 [0.33333333 0.55555556]
 [1.         0.66666667]]
Actual Output:
[[0.92]
 [0.86]
 [0.89]]
Predicted Output:
[[0.77572108]
 [0.76163411]
 [0.78022734]]
-----Epoch- 1 Ends-----

-----Epoch- 2 Starts-----      Input:
[[0.66666667 1.   ]
 [0.33333333 0.55555556]
 [1.         0.66666667]]
Actual Output:
[[0.92]
 [0.86]

```

```

    [0.89]]
    Predicted Output:
    [[0.77772924]
    [0.76355633]
    [0.78226537]]
    ----- Epoch- 2 Ends -----

    ----- Epoch- 3 Starts -----
    Input:
    [[0.66666667 1.          ]
    [0.33333333 0.55555556]
    [1.          0.66666667]]
    Actual Output:
    [[0.92]
    [0.86]
    [0.89]]
    Predicted Output: [[0.77967928]
    [0.76542431]
    [0.78424375]]
    ----- Epoch- 3 Ends -----

    ----- Epoch- 4 Starts -----
    Input:
    [[0.66666667 1.          ]
    [0.33333333 0.55555556]
    [1.          0.66666667]]
    Actual Output:
    [[0.92]
    [0.86]
    [0.89]]
    Predicted Output: [[0.78157364]
    [0.7672403 ]
    [0.78616501]]
    ----- Epoch- 4 Ends -----

    ----- Epoch- 5 Starts -----
    Input:
    [[0.66666667 1.          ]
    [0.33333333 0.55555556]
    [1.          0.66666667]]
    Actual Output: [[0.92]

    [0.86]
    [0.89]]
    Predicted Output:
    [[0.78341464]
    [0.76900642]
    [0.78803156]]
    ----- Epoch- 5 Ends -----

    Input:
    [[0.66666667 1.          ]
    [0.33333333 0.55555556]
    [1.          0.66666667]]
    Actual Output:
    [[0.92]
    [0.86]
    [0.89]]
    Predicted Output:
    [[0.78341464]
    [0.76900642]
    [0.78803156]]

```

RESULT:

Thus the Back propagation Algorithm using ANN was implemented successfully.

Ex. No. : 14**Demonstrate Naïve Bayesian Classifier Machine Learning****PROBLEM STATEMENT:**

Python Program to Implement and Demonstrate Naïve Bayesian Classifier Machine Learning

AIM:

To Demonstrate Naïve Bayesian Classifier Machine Learning.

PROCEDURE:

Step 1: Import the required libraries

Step 2: Load the heart dataset

Step 3: Build the Bayesian model

Step 4: Predict the data

Step 5: Find the accuracy

PROGRAM:

```
import csv
import random
import math

def loadcsv(filename):
    lines = csv.reader(open(filename, "r"));
    dataset = list(lines)
    for i in range(len(dataset)):
        #converting strings into numbers for processing
        dataset[i] = [float(x) for x in dataset[i]]

    return dataset

def splitdataset(dataset, splitratio):#67% training size
    trainsize = int(len(dataset) * splitratio);
    trainset = []
    copy = list(dataset);
    while len(trainset) < trainsize:
        #generate indices for the dataset list randomly to pick ele for training data
        index = random.randrange(len(copy));
        trainset.append(copy.pop(index))
    return [trainset, copy]

def separatebyclass(dataset):
    separated = {} #dictionary of classes 1 and 0

    #creates a dictionary of classes 1 and 0 where the
    #values are#the instances belonging to each class
    for i in range(len(dataset)):
        vector = dataset[i]
        if (vector[-1] not in separated):
            separated[vector[-1]] = []
        separated[vector[-1]].append(vector)

    return separated
```

```

def mean(numbers):
    return sum(numbers)/float(len(numbers))
def stdev(numbers):
    avg = mean(numbers)
    variance = sum([pow(x-avg,2)
    for x in numbers])/float(len(numbers)-1)
    return math.sqrt(variance)
def summarize(dataset): #creates a dictionary of classes
    summaries = [(mean(attribute), stdev(attribute))
    for attribute in zip(*dataset)];del summaries[-1] #excluding labels +ve or -ve
    return summaries
def summarizebyclass(dataset):
    separated = separatebyclass(dataset);
    #print(separated)
    summaries = {}
    for classvalue, instances in separated.items():
    #for key,value in dic.items()
    #summaries is a dic of tuples(mean,std) for each class value
summaries[classvalue] = summarize(instances) #summarize is used to cal to mean and std
    return summaries
def calculateprobability(x, mean, stdev):
    exponent = math.exp(-(math.pow(x-mean,2)/(2*math.pow(stdev,2))))
    return (1 / (math.sqrt(2*math.pi) * stdev)) * exponent

def calculateclassprobabilities(summaries, inputvector):
    probabilities = {} # probabilities contains the all prob of all class of test data
    for classvalue, classsummaries in summaries.items():#class and attribute
information as mean and sd
        probabilities[classvalue] = 1
        for i in range(len(classsummaries)):
            mean, stdev = classsummaries[i] #take mean and sd of every attribute for class 0 and 1
seperately
            x = inputvector[i] #testvector's first attribute
            probabilities[classvalue] *= calculateprobability(x, mean, stdev);#use normal dist

    return probabilities
def predict(summaries, inputvector): #training and test data is passed
    probabilities = calculateclassprobabilities(summaries, inputvector)
    bestLabel, bestProb = None, -1
    for classvalue, probability in probabilities.items():#assigns that class which has he
highest probif bestLabel is None or probability > bestProb:
        bestProb = probability
        bestLabel = classvalue
    return bestLabel
def getpredictions(summaries, testset):
    predictions = []
    for i in range(len(testset)):
        result = predict(summaries, testset[i])
        predictions.append(result)
    return predictions

```

```

def getaccuracy(testset, predictions):
    correct = 0
    for i in range(len(testset)):
        if testset[i][-1] == predictions[i]:
            correct += 1
    return
    (correct/float(len(testset))) *
    100.0
def main():
    filename = '/content/naivedata.csv'
    splitratio = 0.67
    dataset = loadcsv(filename);
    trainingset, testset = splitdataset(dataset, splitratio)
    print('Split {0} rows into train={1} and test={2} rows'.format(len(dataset),
    len(trainingset), len(testset)))
    # prepare model
    summaries = summarizebyclass(trainingset);
    #print(summaries)
    # test model
    predictions = getpredictions(summaries, testset) #find the predictions of test data
    with the training data
    accuracy = getaccuracy(testset, predictions)
    print('Accuracy of the classifier is : {0}%'.format(accuracy))
    if __name__ == "__main__":
        main()
OUTPUT:
Split 767 rows into train=513 and test=254
rows Accuracy of the classifier is :
71.25984251968504%

```

RESULT:

Thus the Naïve Bayesian Classifier Machine Learning was demonstrated and implemented successfully.